University of Leicester

School of Computing and Mathematical Science

Engineering Dynamic Succinct Data Structures

Abdulhakeem Ibrahim

Supervisor: Rajeev Raman

A thesis submitted in partial fulfilment of the University's requirements

for the degree of Doctor of Philosophy

July, 2024

Acknowledgements

In the name of Allah, the Beneficent, the Most Merciful. All praises be to Allah who gave me the strength, wisdom, and guidance to complete this thesis.

This has been a long *six-year* journey, and there are many people I would like to thank. First, I would like to thank my PhD supervisor, Rajeev Raman, for patiently supporting and teaching me over these six years. His mentorship and guidance have allowed me to develop both personally and professionally in ways I never imagined.

A huge thank you to my co-supervisor, Dr. Stanley Fung, for his guidance and invaluable support during this journey. I would also like to thank Thomas Erlebach and Fer-Jan de Vries for their support during my GTA duties and panicky moments. To my fellow PhD researchers in the School of Computing and Mathematical Sciences, thank you for sharing time with me and providing helpful advice during our time together.

I would also like to thank the University of Leicester for the Graduate Teaching Assistantship, which supported my research. Likewise, a huge thank you to the Petroleum Technology Development Fund (PTDF) for the financial support provided to allow me to focus on my research. I am also grateful to the Federal University Birnin Kebbi for the study fellowship approved for me.

Last but not least, I would like to thank my parents, brothers, and sisters for their support and constant prayers. My profound gratitude to my wife for her prayers, support, understanding, and enormous patience with me during this journey.

Abstract

When dealing with big data, where storage and query speed are paramount, traditional data structures are fast but have a huge memory footprint. Succinct structures use space close to the information-theoretic lower bound while remaining competitive in query times compared to traditional ones. Achieving space efficiency while supporting dynamic operations efficiently presents a significant challenge. A common naive approach is to rebuild the static data structure for each update, which becomes very expensive as the data structure grows.

In this work, several innovative dynamic data structures tailored for integer compression are introduced, addressing both the need for compact representation and the ability to handle updates and queries efficiently. The contributions include extensible arrays designed for building append-only Elias-Fano representations that support incremental data additions without requiring complete reconstruction, shown to be competitive with existing solutions.

A practical implementation of the work of Pătraşcu and Thorup [50] on constant time predecessor search in small sets is also provided, which was initially seen as theoretical with no practical implementation. It is demonstrated that this implementation is not only simple but also competitive in performance. Leveraging this implementation, a practical implementation of a dynamic searchable prefix-sum data structure based on the works of Dietz [21] and Raman [52] is proposed. It is shown that these theoretical data structures can now be practically implemented with SIMD instructions widely available on recent architectures.

Contents

1 Introduction **1**

1.1 Overview 1

1.2 Motivation 2

1.3 Research Questions 3

1.4 Contributions 3

1.5 Thesis Organisation 4

1.6 Chapter Summary 5

2 Background and Tools **6**

2.1 Introduction 6

2.2 Basic Concepts 7

2.2.1 Model of Computation 8

2.2.2 Memory Architecture 9

2.2.3 Information Theoretic Lower Bound (ITLB) 11

2.2.4 Empirical Entropy 12

2.2.5 Bit-Vectors 13

2.3 Space-Efficient Data Structures 16

2.4 Fixed and Variable-Length Codes 17

2.5 Integer Compressors 17

2.5.1 Unary and Binary Codes 18

2.5.2 Elias-Gamma (γ) Codes 18
2.5.3 Elias-Delta (δ) Codes 19
2.5.4 VByte Codes 19
2.5.5 Golomb-Rice Codes 19
2.5.6 Packed Encoding 20
2.5.7 Elias-Fano Encoding 20
2.6 Inverted Index Compression 21
2.7 Chapter Summary 22

3 Research Methodology and Theoretical Framework 24
3.1 Research Paradigm: Algorithm Engineering 24
3.2 Research Questions 25
3.2.1 RQ1: Dynamic Space Efficiency 25
3.2.2 RQ2: Small-Set Query Latency 26
3.2.3 RQ3: Update Efficiency in Dynamic Structures 26
3.3 Success Criteria 27
3.4 Verification Framework 27
3.4.1 Invariant-Based Verification 27
3.4.2 Hoare Triple Specification 28
3.4.3 Test Oracle Methodology 28
3.5 Computing Environment 29
3.5.1 Hardware Specifications 29
3.5.2 Software Environment 29
3.5.3 SIMD Capabilities 30
3.6 Experimental Methodology 30
3.6.1 Benchmark Protocol 30
3.6.2 Dataset Generation 31
3.7 Ethical Considerations 31

3.8 Chapter Summary 31

4 Extensible Array Representation 33

4.1 Extensible Array 33

4.1.1 Vector EA 34

4.1.2 Simple EA 35

4.1.3 Brodnik et al EA 36

4.1.4 Collection of Extensible Arrays 37

4.2 Proposed Data Structure 38

4.3 Dynamic Data Structures 41

4.3.1 ITLB Analysis in Dynamic Context 41

4.3.2 Append-Only Bitvector 44

4.3.3 Elias-Fano 46

4.4 Experiments 49

4.4.1 Experimental Setup 49

4.4.2 Append Performance Comparison 49

4.4.3 Space Efficiency Comparison 50

4.4.4 Access Performance 50

4.5 Applications of Extensible Arrays 51

4.5.1 Inverted Index Compression 51

4.5.2 Dynamic Bitvectors in Succinct Data Structures 51

4.5.3 Streaming Data Applications 52

4.5.4 Analysis 52

4.6 Chapter Summary 53

5 Fast Predecessor Search in Small Set 55

5.1 Introduction 55

5.2 Preliminaries 56

5.2.1 Advanced Bit Manipulation Instructions 57
5.2.2 Blind Search 57
5.2.3 Significant Bit Positions 58
5.2.4 Compressed Keys 59
5.2.5 Ajtai's Method 59
5.2.6 Fredman's Method 60
5.2.7 SIMD and Builtin Instructions 60
5.3 Computing Significant Bit Positions 62
5.3.1 Random Datasets 63
5.3.2 B-Tree Sub-Tree Size Datasets 63
5.3.3 B-Tree Splitter Keys Datasets 63
5.4 Optimisations 64
5.5 Implementations Using Built-in Instructions 65
5.5.1 Ajtai's Method 65
5.5.2 Fredman's Method 66
5.5.3 Improved Fredman 67
5.5.4 Experiments 69
5.6 Comparison with Dinklage et al. 70
5.6.1 Overview of Dinklage's Approach 71
5.6.2 Methodological Comparison 71
5.6.3 Key Differences 72
5.6.4 Complementary Techniques 73
5.6.5 Empirical Performance Comparison 73
5.7 Comparison with Classic Predecessor Structures 75
5.7.1 Theoretical Comparison 75
5.7.2 Analysis 76
5.7.3 Space-Time Trade-Off Analysis 77

5.8 Chapter Summary 77

6 Dynamic Searchable Prefix Sum 80

6.1 Preliminaries 80

6.2 Related Work 81

6.3 Searchable Prefix Sum 83

6.4 Dynamic Searchable Prefix Sum 84

6.5 Operations 85

6.6 Optimisations 86

6.7 Chapter Summary 86

7 Conclusions and Future Work 88

7.1 Summary of Contributions 88

7.1.1 Extensible Array Representation (Chapter 4) 88

7.1.2 Fast Predecessor Search in Small Sets (Chapter 5) 89

7.1.3 Dynamic Searchable Prefix Sum (Chapter 6) 89

7.2 Reflective Analysis of the Research Process 90

7.3 Limitations 91

7.4 Future Research Directions 91

7.4.1 Short-Term Extensions 91

7.4.2 Medium-Term Research 92

7.4.3 Long-Term Vision 92

7.5 Concluding Remarks 93

Appendices 101

A Implementation Code Samples 102

A.1 ResizableBitVector Implementation 102

A.2 SIMD Predecessor Search Implementation 104

A.3 Ajtai Method with Lookup Tables 106

A.4 Dynamic Prefix Sum Operations 108

B Benchmark Code and Project Statistics 111

B.1 Project Statistics 111

B.2 Extensible Array Benchmark 113

B.3 Predecessor Search Benchmark 116

B.4 Build and Run Instructions 118

B.4.1 Chapter 4: Elias-Fano (Java/Maven) 118

B.4.2 Chapter 5: Fast Predecessor Search (C++) 119

B.5 Computing Environment 120

B.6 Data Availability 120

List of Figures

4.1 Consecutive blocks of 2^i where $0 \leq i \leq \log n$ 36

5.1 Blind trie for $S = \{7, 85, 102, 200, 205, 232\}$ showing the significant bit positions in the internal nodes and significant bit positions on edges. The leaves hold the index i of $S[i]$ 58

5.2 Number of significant bit positions for various datasets 64

5.3 Time in seconds for 2^{27} *rank* operations on various implementations 70

6.1 Balanced binary tree storing prefix sums of A 82

List of Tables

2.1 All possible bit-strings of length 3 11

2.2 All possible sequences of length 3 that adds up to 5 12

2.3 Documents with their set of terms 21

2.4 Inverted list of terms with the occurrences as document id 22

3.1 Quantified success criteria for proposed data structures 27

3.2 Hardware specifications for experimental evaluation 29

3.3 Software environment specifications 29

4.1 Append 24 to the vector and resize 34

4.2 Worst-case capacity after resize for various resize factors f. The percentages below each factor indicate maximum internal fragmentation. Using $f = n/w$ achieves <2% fragmentation. 39

4.3 Trade-off between growth factor, fragmentation, and space efficiency 43

4.4 Append operation performance comparison 49

4.5 Space efficiency comparison. The thesis `ResizableBitVector` maintains <2% fragmentation vs ArrayList's 18–35%. 50

4.6 Random access performance. The bit-level addressing in `ResizableBitVector` adds 45–65% overhead compared to word-aligned `ArrayList` access. 50

5.1 Instructions used and their descriptions 61

5.2 Result of shifting input of various array sizes by values between $0 - 12000$. . 65

5.3 Comparison of approaches: this work vs Dinklage et al. [22] 71

5.4 Speedup relative to linear search. Higher is better. The Ajtai method achieves 3.5–5.2× speedup over linear search and consistently outperforms STL `std::lower_bound`. 74

5.5 Query time for 2^{25} predecessor queries. The $O(1)$ Ajtai method from this work outperforms Dinklage et al.'s $O(\log \log u)$ B-tree by 3.7–4.5× for small static sets. 75

5.6 Comprehensive comparison of predecessor data structures. Variables: k = set size, b = significant bit positions, u = universe size, n = number of elements, w = word size. 76

5.7 Practical space-time trade-offs. The improved Fredman method offers the best balance of speed and space efficiency for typical use cases. 77

6.1 Array A stored plainly to support *increment/decrement* in $O(1)$ 81

6.2 Array B such that $B[i] = sum(i)$ to support *sum* in $O(1)$ 81

6.3 Array A with original sequence 85

6.4 Array B with possibly out of *sync* prefix sums 85

6.5 SIMD Array C with all the recent updates 85

6.6 Array B after $m = 12$ updates is now refreshed with C 85

B.1 Lines of code by component. Excludes blank lines and comments. Third-party libraries (sux4j, fastutil) are not counted. 112

B.2 Computing environment for experimental evaluation 120

Chapter 1

Introduction

1.1 Overview

Big data refers to the massive volume of structured and unstructured data generated at unprecedented rates, necessitating innovative approaches for storage and processing. Digital devices, social media platforms, and IoT (Internet of Things) sensors contribute significantly to this exponential data growth. Estimates suggest that the global data volume could reach 175 zettabytes by 2025, driven by the increasing digitisation of various sectors including healthcare, finance, security, and manufacturing [18]. This rapid data expansion poses significant challenges for traditional storage solutions, which are often inadequate in terms of capacity and cost-efficiency. Consequently, there is a pressing need for advanced storage technologies such as distributed file systems and cloud storage, which offer scalability and flexibility. Equally important is the development of efficient processing algorithms capable of handling large-scale data analytics and processing. Technologies like Apache Hadoop, Apache Spark, and parallel processing systems are critical in managing and extracting insights from big data. As the volume and complexity of data continue to grow, ongoing advancements in both storage and processing are essential to harness their full potential in

driving innovation and decision-making across various fields. The rate at which data grows necessitates an out-of-box approach to efficient storage and query processing.

Traditional processing methods and algorithms are fast when dealing with such datasets but have high space usage requirements. One of the key challenges is to design data structures that are both space-efficient and time-efficient, meaning they use the least space possible while query operations remain fast. The most common approach is to compress the data by exploiting some of its properties, such as repetitiveness. Pre-processing also helps in speeding up subsequent queries (such as search queries) to the dataset. Compressing large datasets reduces space usage but may slow down queries due to the need to partially or fully decompress the data.

1.2 Motivation

Recent research focuses on ways to efficiently compress data while still allowing queries without full decompression. This approach carries several advantages: (i) reduced space usage, (ii) fitting more data in memory, and (iii) executing queries directly on compressed data. Succinct data structures [38] are a specific class of data structures that aim to solve this problem by using minimal space while still providing fast query times. These structures are particularly useful in applications where memory is at a premium, such as DNA or protein sequencing, search engines, and large-scale text repositories like Wikipedia [47]. Using information theory and data compression, the aim is to provide data structures that can fit into primary memory for processing. The challenge increases when the data is dynamic and requires updates (insertions and/or deletions), as the folklore approach of rebuilding the data structure is not optimal.

Many algorithms for dynamic succinct data structures have been proposed that provide promising theoretical lower bounds, but some are either impractical or too complex to implement. This impracticality may be due to lack of available hardware or software technologies.

With rapid developments in the hardware and software domain, these limitations can change. In this thesis, several of these algorithms are considered, implemented, and improved, with optimisations provided where applicable.

1.3 Research Questions

This thesis addresses the following quantified research questions:

- **RQ1**: Can dynamic succinct data structures maintain space usage within a constant factor of the Information Theoretic Lower Bound while supporting efficient update operations?
- **RQ2**: Can SIMD-based implementations of predecessor search achieve significant speedup over comparison-based approaches on small static sets?
- **RQ3**: Can dynamic data structures support dynamic operations with amortised constant time while maintaining competitive query performance?

1.4 Contributions

A range of dynamic data structures that provide improvements upon existing approaches and implementations is proposed in this thesis. These include Extensible Arrays that can be used in many applications, such as the Extensible Elias-Fano data structure. A detailed analysis of the experiments is also provided. More specifically, the contributions are as follows:

- A simple and efficient algorithm for constructing Extensible Arrays (EA) is presented. Different approaches to the data structure are implemented along with heuristics that optimise them with excellent performance in space and speed. This is utilised to build an Extensible Elias-Fano structure that shows promising improvements over other implementations [47].

- An engineered implementation of fast predecessor search data structure in small sets is provided. This implementation is built using Single-Instruction-Multiple-Data (SIMD) instructions to achieve parallelism. It is simple, practical, and achieves competitive speed and space usage. To the best of available knowledge, this is the first practical implementation of the work of Ajtai [2] on small static integer sets.
- An efficient implementation of a dynamic searchable prefix sum data structure with *increment and search* is provided. This is built on the earlier data structures developed in this thesis and is competitive in speed and space usage.

1.5 Thesis Organisation

The remainder of this thesis is organised as follows:

Chapter 2: Background and Tools introduces the foundational concepts for succinct data structures, including models of computation, memory architecture, Information Theoretic Lower Bounds (ITLB), entropy, bit-vectors, and integer compression techniques.

Chapter 3: Research Methodology and Theoretical Framework presents the Algorithm Engineering research paradigm, defines quantified research questions, establishes success criteria, describes the verification framework, and documents the computing environment.

Chapter 4: Extensible Array Representation presents various implementations of Extensible Arrays (EA). The problem of resizing arrays is addressed, and a more efficient approach is proposed. The new EA is then utilised to build efficient Dynamic Elias-Fano implementations for various problem types.

Chapter 5: Fast Predecessor Search in Small Sets addresses the problem of efficient predecessor search in small sets. An implementation based on Ajtai's [2] work, previously perceived as impractical due to hardware limitations, is provided. Using SIMD instructions,

the first practical implementation of constant time predecessor search on static integer sets is presented.

Chapter 6: Dynamic Searchable Prefix Sum addresses the practical implementation of dynamic searchable prefix sums with *select*. An implementation based on Dietz [21] is presented, along with optimisation methods to improve query performance.

Chapter 7: Conclusions and Future Work presents the key findings and contributions, provides a reflective analysis of the research process, and discusses future research directions and open problems.

1.6 Chapter Summary

This chapter introduced the research context and motivation for the thesis. The challenges of processing and storing big data were presented, along with the need for space-efficient data structures that maintain fast query times. Succinct data structures were introduced as a promising approach to achieving near-optimal space usage while supporting efficient operations.

Three research questions were formulated addressing: (1) maintaining space efficiency close to the Information Theoretic Lower Bound while supporting dynamic operations, (2) achieving significant speedup through SIMD-based predecessor search implementations, and (3) supporting dynamic operations with amortised constant time while maintaining competitive query performance.

The contributions of this thesis were outlined, including extensible arrays with reduced fragmentation, fast predecessor search using SIMD instructions, and dynamic searchable prefix sum implementations. The organisation of the remaining chapters was presented, establishing the structure for the technical contributions that follow.

Chapter 2

Background and Tools

This chapter introduces some key concepts and notation on design and querying of compressed data structures. Starting with the basic concepts, bit-vector which is a key building block of succinct data structures was introduced. Some examples showing how data can be represented using close to the information theoretic minimum was shown and then discussed some well-known techniques and algorithms for compressing integers.

2.1 Introduction

Consider a problem where recording the performances of students in a test is necessary. Whenever a student completes the test, their score is computed and stored in a file. For simplicity, it is assumed that only the score of the students is recorded as they are ready.

$$3, 50, 7, 28, \ldots, 10, 92, 43, \ldots 76, 43, 29.$$

Now that the results have been collected, it will be useful to be able to analyse the query performances. Suppose it is desired to determine how many students failed the test (where the pass mark is at least 40). It is clear that the entire list must be scanned to get an accurate answer. Given $n = 1000000$ be the number of students that took the test, this

means 1000000 scans at best and worst case scenario. This is true for even larger number of students with n number of scans needed to provide answer for such a simple problem.

Pre-processing the data in some way can improve on the earlier solution. If the data is sorted in a particular order, such as ascending order of the scores, the earlier approach remains theoretically the same (same number of scans at worst case), but can perform better empirically (may not scan through always).

$$3, 7, 10, 28, \ldots, 43, 50, 76 \ldots 87, 91, 98.$$

Even better, solving the problem using divide-and-conquer algorithms such as the infamous binary search makes it easier. Solving the problem becomes incredibly easier using binary search. Only the position of the first occurrence of 40 needs to be found and the answer is obtained. This takes about $\log n = \log(1000000) \approx 20$ scans which is ≈ 50000 times faster than the previous approach. This shows that some pre-processing (in this case, ordering the data) can greatly improve some solutions.

Storing the dataset can also be considered. If the same number of bits is naively assigned to each score as a 32-bit or 64-bit integer which is equivalent to a *machine word* (discussed in section 2.2.2), this will need $n \log n$ bits to store the data. But better compression can be achieved by focusing on the frequencies of appearance of each score. This way, scores with higher frequencies will be assigned shorter codes, and those with lower frequencies of appearance can be assigned longer codes. It is clear that the number of bits needed to represent the dataset is $< n \log n$. More discussions on succinct storage is provided in section 2.2.3 and section 2.2.4.

2.2 Basic Concepts

In this section, the various models of computation used throughout the thesis are described. The concept of memory architecture and management, entropy, static and dynamic data

structures is also introduced.

2.2.1 Model of Computation

A model of computation is a mathematical abstraction that describes how a computer algorithm or program operates. There are several different models of computation such as Turing machine, the lambda calculus, and the Random Access Machine (RAM). RAM model is the most common model of computation used in analysis of algorithms.

The Turing machine introduced by Alan Turing is a theoretical machine that is often considered the most powerful model. It operates on a tape with symbols, reading, writing, and moving its head according to a set of rules. Despite its simplicity, it can theoretically simulate any other model of computation.

Lambda Calculus focuses on functions and their application. Instead of manipulating symbols on a tape, it uses expressions to define functions and apply them to arguments. It's a powerful way to reason about logic and computation, forming the foundation of functional programming languages.

The Random Access Machine (RAM) is a model of computation that is based on the concept of a random-access memory. The assumption of this model of computation is that a word of w bits can be manipulated with a single instruction at constant time. Typically, w could be 32 or 64 bits, but there are architectures that supports larger words of 128, 256 or 512 bits. Operations such as addition, subtraction, multiplication, division, *mod* ceiling and floors, etc. are considered as single operations that can be done in $O(1)$ time. Likewise, logic operation such as bitwise *or*, *and*, *xor*, *not* can be done in constant time. These assumptions do not differentiate access time to the instructions due to memory hierarchy as it is generally proven that access to data in lower memory hierarchy is slower than those in higher hierarchy [31]. But in practice, accessing memory on hardware is logarithmic in complexity due to virtual

to physical address translation [40].

Under the RAM model (on which the analysis in this thesis will be based), the number of words is used to compute the space usage as well as time complexity depending on the number of read/write operations of an algorithm. One needs to be careful not to make unrealistic assumptions on the RAM model, for example, a sort operation over a set can be considered as an word-level instruction that can sort a set in constant time [16].

2.2.2 Memory Architecture

To assist with understanding the problems associated with representing and processing data efficiently, the memory architecture of a computer system is described. The hierarchical structure of the computer memory provides balance between speed, capacity, and cost. It comprises multiple levels, each serving distinct roles in data storage and access with higher level of the hierarchy having less space but more speed and the lower level having more space and slower. Typically, data can be stored in the following types of memory [23]:

- Registers: At the top of the memory hierarchy are registers, which are small, fast storage locations within the CPU. Registers hold data that the CPU is currently processing. Their limited size and proximity to the CPU ensure minimal delay in data access, facilitating the execution of instructions at high speed.
- Cache Memory: Cache memory acts as an intermediary between the CPU and the main memory (RAM). It is designed to temporarily store frequently accessed data and instructions, thereby reducing the average time required to access memory. The cache is divided into multiple levels:
  - L1 Cache: Located within the CPU, this is the smallest and fastest cache.
  - L2 Cache: Slightly larger and slower, it is usually situated on the CPU chip.
  - L3 Cache: Even larger and slower, L3 cache is shared among multiple CPU cores

in modern multi-core processors.

The cache leverages spatial and temporal locality to predict and store data that is likely to be used soon, significantly speeding up data retrieval processes.

- Main Memory (RAM) RAM, or Random-Access Memory, is the primary working memory in a computer. It stores data and instructions that the CPU needs while performing tasks. RAM is much faster than disk storage but slower and more expensive per byte than cache memory. It connects to the CPU via a memory bus, which facilitates the transfer of data. Despite its speed, RAM is volatile, meaning it loses all stored data when the computer is turned off.

- Secondary Storage: Secondary storage, such as hard disks and solid-state drives (SSDs), provides non-volatile, long-term data storage. This level of memory is much larger and cheaper per byte than RAM, but significantly slower. Data in secondary storage must be loaded into RAM before the CPU can access it. SSDs, while faster than traditional hard disks, still do not match the speed of RAM or cache.

- Tertiary and Off-line Storage: At the lowest levels of the hierarchy are tertiary and off-line storage solutions, including optical disks, magnetic tapes, and cloud storage. These provide large-scale, cost-effective storage for data that is infrequently accessed. While they offer high capacity, their access speeds are much slower, making them suitable for backup and archival purposes rather than regular data retrieval.

In summary, programs that efficiently utilise fast memory execute much quicker. Specifically, succinct data structures aims to fit more data into the faster cache or RAM, hence minimising the need for time-consuming data transfers from slower secondary storage. This leads to quicker data access and improved overall performance.

2.2.3 Information Theoretic Lower Bound (ITLB)

When dealing with a combinatorial objects, it is quite important to understand the number of bits needed to optimally represent each object. If an object drawn from a set S is to be represented, the minimum amount of bits needed to uniquely represent and identify the object is $\lceil \log |S| \rceil$ bits in the worst case where each object is given codes of same length.. This is referred to as the *information theoretic lower-bound*. Any attempt to use fewer bits to represent the object other than its information theoretic lower bound may not support unique decodability of elements of S. This means more than one object from S may be represented in the same way. However, if more is known about the characteristics of the elements in S, it may be possible to use less than the ITLB.

For example, if S is a set of bit-vectors all having length n. This means that $|S| = 2^n$ which represents all the bit-vectors of length n, and $\lceil \log 2^n \rceil = n$ which matches the ITLB. However, if the assumption is that S contains all bit-vectors with m of their bits set to 1 or 0, then $|S| = \binom{n}{m}$. The ITLB now is $\lceil \log \binom{n}{n} \rceil \approx n \log \frac{n}{m} + O(n) bits$ by using Newton's coefficient formula and Stirling's factorial approximation [55]. Different data structures and their information-theoretic lower bounds are shown below.

Bit-vector: when representing a bit-vector of length n, it has been shown that $|S| = \log 2^n$ possible bit-strings in S with the length of n. For example, there are 2^3 possible bit-strings of length 3, and they are

000	001	010	011	100	101	110	111

Table 2.1: All possible bit-strings of length 3

Proposition 1: The Information theoretic lower bound to represent a bit string of length n is $\lceil \log 2^n \rceil = n$ bits.

Remark: At worst case, writing out a bit-string as it is represents the ITLB.

Binary trees: A binary tree is rooted tree with each node having a maximum of 2 children; left and right child or none. There are $C_n = \frac{1}{n+1}\binom{2n}{n}$ possible variations of binary trees where n is the number of nodes. For $n = 3$, this gives $C_3 = \frac{1}{3+1}\frac{6!}{3!.3!} = 5$

Proposition 2: The information theoretic lower bound to represent a binary tree with n nodes is $2n - O(\log n)$

Prefix sum: Given a set containing sequences of n increasing positive integers that sums up to m with the assumption that m and n are known in advance, it can be shown that the variations of these sequences are $|S| = \binom{m-1}{n-1}$. Using an example, if $n = 3$ and $m = 5$, this gives $\binom{7!}{2!.3!} = 6$

Proposition 2: The information theoretic lower bound to represent prefix sum is $\lceil \log |S| \rceil \leq n \log(\frac{m}{n}) + n \log e$ bits.

[1,2,5]	[1,3,5]	[1,4,5]	[2,3,5]	[2,4,5]	[3,4,5]

Table 2.2: All possible sequences of length 3 that adds up to 5

2.2.4 Empirical Entropy

The concept of Information theoretic lower bound was introduced that provides the least number of bits needed to represent an item from a sequence in the worst case scenario. This means that no prior knowledge on the data to be stored is available, hence the analysis is performed at worst-case. But if some information about the data to be stored is available, it is easier to explore and exploit some properties of the data such that less than the Information theoretic lower bound can be used while still maintaining unique decodability.

Entropy is the lower bound of the number of bits needed to uniquely represent an object in a set. Given the problem of encoding an a set of elements A over the alphabet $\sum =$

$\{a_1, a_2, \ldots a_n\}$ with the occurrence of any $a_i \in A$ represented as $|a_i|$. The probability of a_i in A is given by $p(i) = |a_i|/|A|$. The $0th$ order empirical entropy of A is $H_o(A) = \sum p(i) \log 1/p(i)$. This is called the $zero-order$ or $memory-less$ entropy because the object a_i emitted by the source is independent of all previously emitted objects $a_{i-1}, a_{i0-2}, \ldots a_0$. An example is given here with a bit-string of size n. If the bit-string has 50% of $1s$, then the $H_0 = 0.5 \log 0.5 + 0.5 \log 0.5 = 1$. Here, exactly n bits are needed to represent the object. In another example with the bit-string having 70% of $1s$, then $H_0 = 0.7 \log 0.7 + 0.3 \log 0.3 = 0.8816$ bits. It can now be seen that by simply knowing the frequencies of the alphabets, lower than the ITLB can be used to represent the bit string using $0.8816n$ bits only.

It gets more interesting if the source emitting these symbols has some *memory*, where the probability of a symbol to be emitted depends on the appearance of that symbol in the set. This is called the $higher - order$ entropy or $k^{th} - order$ entropy [44]. The probability of a_i given than $a_0 \ldots a_{n-1}$ has been emitted is given by $p(a_i) = p(a_i|a_0 \ldots a_{n-1})$. Hence, the $k^{th} - order$ entropy $H[a_0 \ldots a_{n-1}] = \sum p(a_0 \ldots a_{n-1}) H[a_0 \ldots a_{n-1}]$.

2.2.5 Bit-Vectors

Bit-vectors are very important building blocks of succinct data structures representing sets, strings, and sequences. A bit-vector is often referred to as bitmap or bit-array in various literature. Henceforth, a bit-vector is assumed to be a sequence of bits that supports *access*, *rank* and *select* operations.

Definition: Bit vector is a sequence $B[0, n-1] \in 0, 1^n$ of length n that supports the following operations:

- $access(B, i)$: returns the bit $B[i]$ for $0 \leq i \leq n-1$.
- $rank_1(B, i)$: returns the number of occurrence of $1s$ in $B[0, i)$. This can be written as $\sum_{j=0}^{n-1} B[j]$ for $0 \leq i \leq n-1$.

- $rank_0(B, i)$: returns $i - rank_1(i)$ for $0 \leq i \leq n-1$.
- $select_1(B, i)$: returns the position of the ith occurrence of 1 in B. This can be written as $max\{j|rank_1(i) = i\}$.
- $select_0(B, i)$: returns the position of the ith occurrence of 0 in B. This can be written as $max\{j|rank_0(j) = i\}$.

Naive Approach

By storing the bit-vector in plain format as sequence of n bits, *access* operation can be supported in *constant* time. Using an additional look-up table T_r such that $T_r[i]$ stores the number of $1s$ till i allows *rank* operations to be supported in constant time. But T_r takes additional $n \log n$ space. To support *select* in constant time, look-up tables T_{s0} and T_{s1} are also used, which store the positions of all $1s$ and $0s$ respectively. Clearly, this approach is time efficient as it supports all operations in constant time, but it is not *space-efficient* as it needs $n + O(n \log n)$ bits of space to achieve this.

Example

- *bit-vector*: 1011001110001001
- $n = 16$
- T_r: the look-up table to support $rank_1$ operation

1	1	2	3	3	3	4	5	6	6	6	6	7	7	7	8
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

- T_{s1}: the look-up table to support $select_1$ operation

1	3	4	7	8	9	13	16
1	2	3	4	5	6	7	8

- T_{s0}: the look-up table to support $select_0$ operation

2	5	6	10	11	12	14	15
1	2	3	4	5	6	7	8

To get $rank_1(6)$, $T_r[6]$ is simply looked up, which returns 3. That means there are 3 set bits from the beginning of the bit-string to position 6. For $rank_0(6)$, the calculation is simply $i - rank_1(6) = 3$. To calculate $select_1(6)$, $T_{s0}[6] = 9$ is looked up. This means the 6th set bit is at position 9 in the bit-string. For $select_0(6)$, $T_{s0}[6] = 12$ is looked up.

An improvement on the above can be made by not storing all the $rank_1[i]$ in T_r, but after every $\log^2 n$ positions. This means T_r occupies $n/\log n$ space. To support $rank_1$, only one entry in T_r needs to be read, accessing at most $O(\log n)$ bits. *select* is also supported by doing *binary-search* over the values of *rank*. This takes $n + O(n/\log n)$ space and the operations take $O(\log n)$ time which is better than the naive approach, but not *optimal*.

Succinct Solution

The bit-vector is divided into *sub-blocks* of $\log n$ bits and *super-blocks* of $\log^2 n$ bits. For each *super-block*, the number of set-bits of all *sub-blocks* to the *super-block* is stored. As each *super-block* is $\log n$ in size, this requires $O(\log n.n/\log^2 n) = o(n)$ bits. For each *sub-block*, the partial *rank* that resets at the beginning of each *super-block* is stored. This requires only $\log\log^2 n/\log n = o(n)$ bits [43].

Example

- *bit-vector*: 10110011100010011011101010001011
- $n = 32$
- *super-block* size: $\log n = 5$
- *sub-block* size: $\log^2 n = 25$

For the purpose of this example, the memory word is assumed to be $w = \log n = 5$.

bit-vector	10110	01110	00100	10001	11000	00010	11
super-blocks	0					11	
blocks	0	3	6	7	9	0	1

To answer $rank_1(i)$, the *rank* values in the *super-block* and *sub-block* containing i are summed. As these store the *global* and *local rank* respectively, only the exact number of bits set inside the *local* block up to position i needs to be determined. This can be done in constant time using *mask* and *popcount* on $\log n$ bits (single *sub-block*) or at most $2 \log n$ bits (intersecting *sub-blocks*). Other implementations of of *rank* and *select* can be found in [6, 20, 45, 47, 49]

2.3 Space-Efficient Data Structures

Space-efficient structures are designed to minimise storage requirements while allowing queries to remain fast. The main challenge for these data structures is to balance between keeping query time as low as traditional data structures, but using space close to *Information-theoretic lower bound*. While most solutions for these data structures are build to be *static*, a lot of research is being done on making them *dynamic* to support update queries.

Succinct data structures were first discussed in Jacobson's Doctoral thesis as a special class of compressed data structures [38]. While compressed data structures may need $O(n)$ additional bits to store an object that requires at least n bits to represent, succinct data structures will only need $o(m)$ additional bits. Unlike traditional data structures that may use standard space storage, compact structures aim to approach the information-theoretic lower bound (ITLB) of space required to store data, otherwise called the *succinct bound*. Trees and text data can be stored compactly using special encoding and application-specific designs. Using a sequence A on n elements as an example, a compact data structure will use $n + O(n)$. On the other hand, succinct data structures are highly-space efficient with the aim of fitting more data into memory to achieve very fast queries (typically constant time). The space usage

is typically $n + o(n)$. Theoretically, these bounds are promising as they can be considered negligible under the asymptotic analysis. However, this may not always be the case in practice and theory may not always translate to practice [32, 31].

2.4 Fixed and Variable-Length Codes

Variable-length codes and *fixed-length* codes are two fundamental approaches to encoding data, particularly integers. Fixed-length codes assign a uniform number of bits to each possible symbol in the set, ensuring simplicity and ease of decoding. However, this often results in inefficiency when encoding symbols with varying probabilities of occurrence. In contrast, variable-length codes assign shorter bit sequences to more frequently occurring symbols and longer bit sequences to less frequent ones [44]. This adaptive allocation optimises the overall bit usage, reducing average code length compared to fixed-length codes, especially when dealing with data with varying entropy. An example is shown below

- *Sequence A*: a b c d
- *fixed-length codes*: a: 000 b:001 c:010 d: 011

Assuming the objects in A have varying frequencies, then variable-length codes may be used to improve on the encoding.

- *Sequence A*: a (60%) b (30%) c (3%) d 2%)
- *variable-length codes*: a: 0 b:1 c:010 d: 011

2.5 Integer Compressors

Integer compressors are designed to compress sequences of integers in a way that minimises storage space while allowing efficient access, retrieval, and sometimes modification of the compressed data. As most of the data structures designed in this thesis are built on integer

compression, an overview of some of the integer list compression algorithms is given. The main aim is to ensure that the shortest uniquely-decodable, variable-length codes are assigned to each integer so that the sequence is encoded with minimal number of bits.

2.5.1 Unary and Binary Codes

Unary codes represents a using a sequence of *'0's* followed by a single *'1'*. The number of *'0's* indicates the value being encoded. For example, the unary encoding of the number 3 is 0001. Unary encoding is simple and intuitive but becomes inefficient for larger numbers due to its linear growth in length with the encoded value.

$$u(x) = 0^{x-1}1$$

One can also use the reversed unary with $u(x) = 1^{x-1}0$ and encode 3 as 1110. Binary codes $B(x)$ represents numbers using the *base-2* numeral system, where each digit (bit) in the sequence represents a power of 2. In binary encoding, numbers are expressed using only *'0's* and *'1's*. This method is highly efficient for computers because it directly corresponds to the hardware's binary logic. For example, $B(3)$ will be 11.

2.5.2 Elias-Gamma (γ) Codes

This method encodes in unary the length of the binary representation of —x— and then encodes x in binary without its highest bit. Hence, $\gamma(x) = u(|x|)[x]_{|x|-1}$ where $[x]_l$ is the least significant bit of x. It can be shown that $|\gamma(x)| = 2|x| - 1 = O(\log x)$ [25]. Assuming the numbers to be encoded fit into a computer word, γ codes for small integers can easily be decoded in $O(1)$ by either using a small *look-up* table or using machine instructions to find the *highest-bit* of x. For example, $\gamma(39) = 11111000111$ where $B(39) = 100111$ with length of 6 bits, then the $U(6) = 1111110$. Concatenating the *unary* and stripping off the leading 1 from the *binary*, the result is 11111000111

2.5.3 Elias-Delta (δ) Codes

While *Elias-γ* codes are efficient for smaller numbers, they can be inefficient for larger integers due to the unary prefix. Elias-Delta coding improves on this by replacing the unary prefix with a *gamma code* of the length of the binary representation. For instance, $\delta(113) = 001111110001$, where the last part 1110001 is the binary representation of 113, prefixed by $\gamma(7) = 00111$. The decoding of delta codes follows naturally from the decoding of gamma codes.

2.5.4 VByte Codes

This method works really well for large numbers. The aim is to speed up decoding by reading fixed-aligned data chunks. Here, integers are represented using a multiple of a fixed number of bits, this can be *a nibble, byte, or even word*. Variable-byte [56] splits each integer into groups of *7 bits*. Each group is stored in one byte. The most significant bit (MSB) of each byte is used as a continuation bit, indicating whether the byte is the final byte of the current integer (continuation bit = 0) or if there are more bytes to come (continuation bit = 1). This method ensures that smaller integers use fewer bytes, though not very suitable for integers that needs less than a byte. For example, $VB(913) = 1110010\underline{1}00000001$ with the 8th bit (control bit) set to 1 showing that the next byte needs to be read to decode the integer. The 2nd byte also had to be padded with zeros to make it a whole byte.

2.5.5 Golomb-Rice Codes

In the early 1970s, Robert [53] introduced Rice coding, a parameterised compression scheme for integers that is particularly effective when the values are clustered around a common value. Rice coding is a special case of Golomb coding and is defined by a parameter k. The Rice code $R_k(x)$ is composed of two parts: the quotient $q = \lfloor (x-1)/2^k \rfloor$ and the remainder $r = x - q \cdot 2^k - 1$. The quotient is encoded in unary, and the remainder is encoded in binary

using k bits. For example, with $k = 5$, $R_5(113) = 000110000$, where the quotient 3 is encoded as 0001 and the remainder 16 as 00000. Using $k = 6$, the code becomes $R_6(113) = 01110000$, which is more efficient because 2^6 is closer to 113 than 2^5. Decoding involves counting the leading zeros to determine the quotient, then calculating the value by multiplying the quotient by 2^k and adding the remainder.

2.5.6 Packed Encoding

Packed encoding methods improve compression ratio and decoding speed by encoding blocks of integers instead of individual values. Binary packing, for instance, represents a block of integers using the minimum number of bits necessary for the largest integer in the block. Simple strategies like Simple-9 and Simple-16 pack integers into 32 or 64-bit words using a few selector bits to indicate how many integers and their bit sizes are packed. Simple-9 uses 4 selector bits and 28 data bits, allowing configurations like 28 1-bit integers or 14 2-bit integers. Simple-8b uses 64-bit words with 4-bit selectors. The QMX method improves on this by packing integers into 128-bit words and storing selectors separately, compressing them with run-length encoding (RLE) for efficiency [44].

2.5.7 Elias-Fano Encoding

Elias-Fano encoding, proposed independently by Elias[24] and Fano[26] in the 1970s, is effective for compressing sorted integer sequences. Each integer in the sequence is split into low bits and high bits. The low bits are stored explicitly, while the high bits are encoded in a bitvector using unary coding. For example, given a sequence $S(n, u)$, each integer $S[i]$ is divided into low bits ℓ_i and high bits h_i, with the high bits stored in a bitvector such that the i-th position is set to 1 at $h_i + i$. This method efficiently represents both the high and low parts of the integers, balancing compression and decoding speed. More discussion on Elias Fano encoding is given in Section 4.3.3.

2.6 Inverted Index Compression

Search engines like databases, store documents in their repository. An index is maintained to allow for faster queries and retrievals. Consider an example with the following web pages (henceforth referred to as documents) containing some terms/words :

- *Doc1* → (book, car, house, money)
- *Doc2* → (cat, money)
- *Doc3* → (car, cat)
- *Doc4* → (book)
- *Doc5* → (money, house)

Table 2.3: Documents with their set of terms

This is very similar to the index at the beginning of books or even this thesis, showing for each page, what terms (topics or sub-topics in this case) can be found in them. In large web systems, query operations across web documents comes with obstacles. The term(s) being queried could occur in very large number of documents with each document having number of terms. The index needs to be optimised in a way that relevant documents to the search terms can be retrieved and returned quickly. Using an inverted index, terms are mapped to their occurrences in a collection of documents, typically represented as a list of document identifiers called *posting list*. However, due to their potentially large size, especially in extensive document collections (like the world-wide-web), it is essential to employ compression techniques to reduce storage requirements and enhance query performance [13, 41]. An inverted index of Table 2.3 is shown below:

- $book \rightarrow (1, 4)$
- $car \rightarrow (1, 3)$
- $house \rightarrow (1, 5)$
- $money \rightarrow (1, 2, 5)$
- $cat \rightarrow (1, 3)$

Table 2.4: Inverted list of terms with the occurrences as document id

Inverted index data structure are very useful in supporting full-text search engines by mapping each term to a list of documents in which it appears to perform rapid searches, retrieve relevant documents, and rank them efficiently based on various algorithms [47]. Likewise in Information Retrieval (IR) systems, inverted indexes are used to quickly locate documents that contain specific words or phrases, making them indispensable for digital libraries, academic databases, and other large text repositories [41]. Moreover, Version Control systems like Git use inverted indexes to manage and retrieve different versions of source code efficiently. By indexing changes and differences between versions, these systems can quickly show the history of changes, compare different versions, and merge updates from multiple contributors.

Given their usefulness and the large amount of data they hold, it is important that they benefit from compressed or even better, succinct data structures. This would mean more data can be loaded in memory for faster processing. Some of these algorithms have been discussed in Section 2.5. Further reading on inverted index compression can be found in [3, 4, 47, 62]

2.7 Chapter Summary

This chapter presented the foundational concepts and tools necessary for understanding succinct data structures. The key topics covered include:

- **Models of Computation**: The Random Access Machine (RAM) model was introduced as the computational framework used throughout this thesis, where word-level operations are assumed to execute in constant time.
- **Memory Architecture**: The hierarchical structure of computer memory (registers, cache, RAM, secondary storage) was described, highlighting the importance of fitting data into faster memory levels for improved performance.
- **Information Theoretic Lower Bound (ITLB)**: The minimum number of bits required to uniquely represent objects from a set was defined, providing the theoretical baseline against which space efficiency is measured.
- **Empirical Entropy**: Zero-order and higher-order entropy were introduced as measures of the minimum bits needed when frequency information about the data is known.
- **Bit-vectors**: The fundamental building block for succinct data structures was presented, including the rank and select operations that enable efficient queries.
- **Integer Compression**: Various encoding schemes (unary, binary, Elias-gamma, Elias-delta, VByte, Rice, and Elias-Fano) were reviewed, providing the compression techniques used in subsequent chapters.
- **Inverted Index Compression**: The application of integer compression to search engine indexing was discussed, demonstrating the practical relevance of the techniques presented.

These concepts form the theoretical foundation for the data structures developed in subsequent chapters, particularly the extensible array representation (Chapter 4), predecessor search (Chapter 5), and dynamic prefix sum (Chapter 6).

Chapter 3

Research Methodology and Theoretical Framework

This chapter presents the research methodology employed in this thesis, defines quantified research questions, establishes success criteria, and describes the verification framework used to validate the proposed data structures. The computing environment specifications are also documented to ensure reproducibility of experimental results.

3.1 Research Paradigm: Algorithm Engineering

The research presented in this thesis follows the *Algorithm Engineering* paradigm [54], which bridges the gap between theoretical algorithm design and practical implementation. Unlike traditional algorithm analysis that focuses primarily on asymptotic complexity, algorithm engineering emphasises the following iterative cycle:

1. **Design**: Development of algorithms based on theoretical foundations and complexity analysis.
2. **Analysis**: Formal verification of correctness and derivation of theoretical bounds.

3. **Implementation**: Translation of algorithms into efficient code, considering hardware characteristics.

4. **Experimentation**: Empirical evaluation using realistic workloads and datasets.

5. **Refinement**: Iterative improvement based on experimental insights.

This methodology is particularly appropriate for succinct data structures, where theoretical bounds often involve constants that significantly impact practical performance. The gap between theoretical models (such as the RAM model) and actual hardware behaviour (cache hierarchies, branch prediction, SIMD capabilities) necessitates empirical validation of theoretical claims.

3.2 Research Questions

Three quantified research questions guide the investigations presented in this thesis:

3.2.1 RQ1: Dynamic Space Efficiency

Can dynamic succinct data structures maintain space usage within a constant factor of the Information Theoretic Lower Bound (ITLB) while supporting efficient update operations?

Quantified Formulation: Given a dynamic data structure D storing n elements from a universe of size u, the space requirement $S(D)$ should satisfy:

$$S(D) \leq c \cdot \mathrm{ITLB}(n, u) + o(\mathrm{ITLB}(n, u)) \tag{3.1}$$

where c is a small constant and the $o(\cdot)$ term represents structural overhead that becomes negligible for large n.

For monotone integer sequences using Elias-Fano encoding, the ITLB is:

$$\text{ITLB}(n, u) = n \cdot \lceil \log_2(u/n) \rceil + 2n \text{ bits} \tag{3.2}$$

3.2.2 RQ2: Small-Set Query Latency

Can SIMD-based implementations of predecessor search achieve constant-time query latency on small static sets, and by what factor do they outperform traditional approaches?

Quantified Formulation: For a set S of k integers where $k \leq 256$, predecessor queries should be answerable in:

$$T_{\text{query}}(k) = O(1) \text{ with } T_{\text{query}} < T_{\text{binary}} \tag{3.3}$$

where T_{binary} is the query time of optimised binary search. The speedup is achieved through parallel comparison using SIMD registers.

3.2.3 RQ3: Update Efficiency in Dynamic Structures

Can dynamic data structures support dynamic operations with amortised constant time while maintaining competitive query performance?

Quantified Formulation: For a dynamic structure supporting updates, the amortised cost of n operations should satisfy:

$$\sum_{i=1}^{n} T_{\text{update}}(i) \leq c' \cdot n \tag{3.4}$$

where c' is a small constant independent of n. Additionally, query operations (access, rank, select, nextGEQ) should remain within a constant factor of static implementations.

3.3 Success Criteria

The success of the proposed data structures is evaluated against the following quantified criteria:

Criterion	**Target**	**Measurement Method**
Space efficiency	Constant factor of ITLB	Bits per element ratio
Query latency (small sets)	Faster than binary search	Nanoseconds per query
Update latency (dynamic)	Amortised $O(1)$	Total time / n operations
Correctness	100% pass rate	Automated test suite
Reproducibility	$< 5\%$ variance	Standard deviation over 20+ runs

Table 3.1: Quantified success criteria for proposed data structures

3.4 Verification Framework

Correctness verification is essential for data structures where subtle implementation errors can produce incorrect results that may go undetected. The verification approach employed in this thesis consists of three complementary methods:

3.4.1 Invariant-Based Verification

Each data structure maintains specific invariants that must hold after every operation. These invariants are formally specified and verified through assertions in the implementation.

Extensible Array Invariant: For an extensible array A with logical size n and physical capacity c:

- $0 \leq n \leq c$
- All elements at indices $0, 1, \ldots, n-1$ are valid and accessible
- After `append(x)`: $A[n'] = x$ where $n' = n + 1$

Elias-Fano Invariant: For an Elias-Fano structure storing monotone sequence S:

- $S[i] \leq S[i+1]$ for all $0 \leq i < n-1$ (monotonicity)
- $\text{select}_1(\text{high_bits}, i) - i = \lfloor S[i]/2^\ell \rfloor$ (high-bit correctness)
- $\text{low_bits}[i] = S[i] \mod 2^\ell$ (low-bit correctness)

3.4.2 Hoare Triple Specification

Critical operations are specified using Hoare triples $\{P\}$ operation $\{Q\}$, where P is the precondition and Q is the postcondition.

Example - GROW() operation:

$$\{\text{valid}(A) \wedge n = c \wedge \forall i \in [0, n) : A[i] = v_i\}$$

```
GROW()
```

$$\{\text{valid}(A) \wedge c' > c \wedge n' = n \wedge \forall i \in [0, n) : A[i] = v_i\}$$

This specification ensures that the GROW operation increases capacity while preserving all existing elements and their values.

3.4.3 Test Oracle Methodology

Implementations are validated against reference implementations (test oracles) that prioritise correctness over efficiency. For each proposed data structure:

1. A naïve reference implementation is developed using standard library containers
2. Random test sequences are generated with controlled parameters
3. Results from the optimised implementation are compared against the oracle
4. Discrepancies trigger detailed debugging and invariant checking

3.5 Computing Environment

All experiments reported in this thesis were conducted on the following hardware and software configuration:

3.5.1 Hardware Specifications

Component	**Specification**
Processor	Intel Core i5-4460 @ 3.40GHz
Architecture	x86_64 (64-bit)
SIMD Support	SSE4.2, AVX, AVX2, BMI1, BMI2
Memory	8 GB DDR3
L1 Cache (per core)	32 KB data + 32 KB instruction
L2 Cache (per core)	256 KB
L3 Cache (shared)	6 MB

Table 3.2: Hardware specifications for experimental evaluation

3.5.2 Software Environment

Component	**Version**
Operating System	Ubuntu 22.04.3 LTS
Kernel	Linux 5.15.0
Java Runtime	OpenJDK 11.0.20
C++ Compiler	g++ 11.4.0
Build System (Java)	Apache Maven 3.9.x
Optimisation Flags	-O3 -march=native

Table 3.3: Software environment specifications

3.5.3 SIMD Capabilities

The experiments in Chapter 5 utilise Intel AVX2 SIMD instructions with 256-bit registers, which enable parallel processing of multiple data elements in a single instruction. The processor supports the following instruction set extensions:

- **AVX2**: 256-bit integer and floating-point SIMD operations
- **BMI1/BMI2**: Bit manipulation instructions including PEXT, PDEP, LZCNT, and TZCNT
- **SSE4.2**: String and text processing instructions

These capabilities are essential for the constant-time predecessor search implementations described in Chapter 5, following the methodology established by Dinklage et al. [22].

3.6 Experimental Methodology

3.6.1 Benchmark Protocol

To ensure statistical validity and reproducibility, all experiments follow a standardised protocol:

1. **Warm-up Phase**: Execute 5 warm-up iterations to stabilise JIT compilation (Java) or cache state
2. **Measurement Phase**: Execute minimum 20 timed iterations
3. **Isolation**: Pin experiments to performance cores where possible; disable background processes
4. **Randomisation**: Use fixed random seeds for reproducibility; document seeds in supplementary materials

5. **Statistical Reporting**: Report mean, standard deviation, and 95% confidence intervals

3.6.2 Dataset Generation

Experimental datasets are generated synthetically with controlled parameters:

- **Monotone sequences**: Generated by cumulative sum of random gaps drawn from specified distributions (uniform, exponential, Zipfian)
- **Set sizes**: Varied across $n \in \{10^3, 10^4, 10^5, 10^6, 10^7\}$
- **Universe sizes**: Tested with u/n ratios from 1.1 to 1000
- **Query workloads**: 2^{25} random queries per configuration

3.7 Ethical Considerations

The research presented in this thesis involves only computational experiments on synthetic data. No human subjects, personal data, or sensitive information were used. All software dependencies are open-source with compatible licences (Apache 2.0, MIT, BSD). The experimental code and data are available for academic reproducibility.

3.8 Chapter Summary

This chapter established the methodological foundation for the research presented in this thesis. The Algorithm Engineering paradigm provides a framework for bridging theoretical analysis and practical implementation. Three quantified research questions (RQ1–RQ3) guide the investigations, with explicit success criteria enabling objective evaluation. A comprehensive verification framework ensures correctness through invariant checking, Hoare triple specifications, and test oracle comparisons. The computing environment has been docu-

mented to support reproducibility, and the experimental methodology ensures statistical validity of results. Subsequent chapters apply this methodology to specific data structures: extensible arrays (Chapter 4), fast predecessor search (Chapter 5), and dynamic searchable prefix sums (Chapter 6).

Chapter 4

Extensible Array Representation

This chapter presents existing work in the area of extensible arrays. Dynamic arrays that support append-only operations alongside the standard static operations are introduced. The focus is then placed on extensible arrays of non-standard width ($1 \leq width \leq w$). These dynamic arrays are subsequently utilised to build an efficient Elias-Fano implementation that supports *append-only* operations. Variations of this data structure that perform well in various settings are presented.

4.1 Extensible Array

Arrays are excellent example of static data structures that have been studied extensively [44]. Vector is the most basic form of extendible array that supports random access in constant time. They have been shown to solve a number of dynamic data structures problem such as randomised queues, stacks, priority queues and queues [11, 44, 12], and have been used in building complex compact data structures [42, 51]. An Extendible Array (EA) built upon the idea of vectors is described in this chapter. Other variants of EAs of non-standard widths (such as bit-vectors, shorts, characters, etc.) are also presented. The operations supported are:

- $create(n)$: Create an array of initial size n
- $access(i)$: Access the item at index i.
- $append(x)$: Adds a new item at the end of array.
- $grow()$: resize the array to new size $m > n$

4.1.1 Vector EA

A Vector is a common example of extendible array that supports *access* operation in $O(1)$ time and $grow/shrink$ in amortised constant time using $n + O(n)$. To append a new item to a vector of size n that is already full, we need to increase it's size. The most common approach is to double the size of the vector to $2n$ to allow for up to n new appends before the need to resize the vector. This may create internal fragmentation problem of $n - 1$ of a vector of $2n$. Likewise, using first-fit allocator, a sequence of $grow()/shrink()$ could lead to inefficiencies such that despite the array using n space, the allocated range of memory could span $\Theta(n \log n)$. A typical vector supports only word-size elements, but we are also interested of elements of non-standard width.

Algorithm 1 Grows A by n

```
procedure GROW()
A_o is new array of size 2n
for i := 1 to size(A) do
    A_o[i] := A[i]
end for
return A_o
```

Example

14	56	32	

14	56	32	24				

Table 4.1: Append 24 to the vector and resize

Correctness of GROW Operation

The correctness of the GROW operation is established using a Hoare triple specification.

Precondition (P): Let A be a valid array with logical size n and physical capacity c, where $n = c$ (array is full). Let $v_0, v_1, \ldots, v_{n-1}$ denote the current values stored in A.

Postcondition (Q): After GROW completes, the resulting array A_o satisfies:

1. $\text{capacity}(A_o) = 2n > c$ (capacity has increased)
2. $\text{size}(A_o) = n$ (logical size unchanged)
3. $\forall i \in [0, n) : A_o[i] = v_i$ (all values preserved)

Hoare Triple: $\{P\}$ `GROW()` $\{Q\}$

Proof: By loop invariant analysis.

- *Initialisation*: Before the loop, A_o is allocated with capacity $2n > c$. No elements have been copied yet.
- *Maintenance*: At iteration i, the assignment $A_o[i] := A[i]$ copies element v_i to position i in the new array. The loop invariant is: $\forall j \in [1, i] : A_o[j] = v_j$.
- *Termination*: When $i = \text{size}(A) = n$, all elements have been copied. The postcondition Q is established.

Termination: The loop iterates exactly n times, so GROW terminates in $O(n)$ time.

4.1.2 Simple EA

By choosing a fixed resize factor f, the internal fragmentation from above can be reduced. Normally, f is a power of 2. This keeps the $O(1)$ access and append time and worst case $O(n)$ amortised time for grow operation. The worst case internal fragmentation is improved to $O(f)$ but speed of grow operation is significantly degraded.

4.1.3 Brodnik et al EA

Brodnik [11] proposed a singly extendible array that use $n+O(\sqrt{n})$ space at all times (where n is the total number of elements in the array). They achieve this by dividing the array into consecutive *data−blocks* DB_A of the sizes 2^i where $0 \leq i \leq \log n$. A conceptual *super−block* SB_A of size $2^{\log n}$ contains *data-blocks* of the same size. As each data block is of size $2^{\lceil i/2 \rceil}$, a *super-block* SB_i will have a maximum of $2^{\lfloor i/2 \rfloor}$ data blocks until it is full. Therefore, SB_i will have a total of 2^i elements in it. To further keep space usage low, memory is only allocated for non-empty blocks.

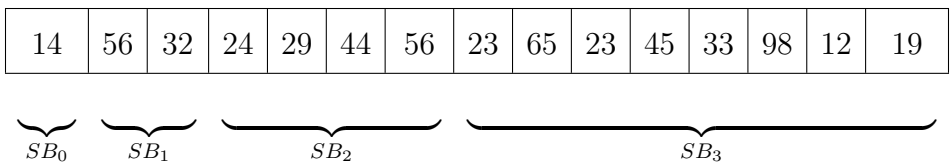

Figure 4.1: Consecutive blocks of 2^i where $0 \leq i \leq \log n$

The algorithm for access operation is shown below. A key aspect of it is how to compute the position of the leading set bit in the binary representation of $i+1$ in constant time. Brodnik [12] gives a method that uses basic arithmetic and logic operators to achieve this in constant time. But modern machines allows for this as well using Bit Scan Reverse instruction [17].

Algorithm 2 Access(i)

1: Let r be the binary representation of $i-1$ with no leading zeros
2: Let the *super-block* $k = \lfloor \log i + 1 \rfloor$.
3: Let $b = \lfloor k/2 \rfloor$ bit of the *super-block* after the leading 1-bit
4: Let e be the remaining $\lfloor k/2 \rfloor$ bits of r.
5: **return** the element e in *data-block* b of *super-block* k.

As all the steps in *access* can be done in constant time if we ignore the cost of dynamic memory allocation, the running time is $O(1)$. Space wasted from internal fragmentation is $O(\sqrt{n}$ words which they show to be optimal.

4.1.4 Collection of Extensible Arrays

Joannou [39] described various dynamic random access data structures built as Collection of Extendible Arrays (CEA), which are essentially, a vector of pointers to other vectors. A vector of size n is divided into consecutive blocks of fixed size $b > 1$, and records are stored in fixed-size data blocks of size b each. A vector of n/b pointers (called the index block) stores a pointer to each of the data-blocks. To perform $access(i)$ operation, we only fetch the $(i \mod b) - th$ entry in the $\lfloor i/b \rfloor - th$ data block. It is clear that access operation is $O(1)$ (though, could be slower due to 2-random accesses), and $O(1)$ amortised constant time for grow operation. This approach clearly reduces the internal fragmentation, though, the choice of b needs to be carefully made to avoid having $\theta(b)$ wasted space per data block.

Another approach is to vary the size of the blocks as done in [11]. The vector of size n is divided into super blocks of size $1, 2, 3 \ldots, 2^{\lfloor \log_2(n) \rfloor}$. And each super block is sub-divided into data blocks of equal sizes. The access operation takes constant time, while grow operations takes $O(1)$ amortised time. The wasted space is $O\sqrt{n}$ words. The modified Brodnik shares same analysis, though handles the data structure a bit differently. All data blocks are of same initial size $b = 2$. A grow operation adds an element to the last data block, and when full, we either double the index or copy two existing data blocks into a newly created data block of size $2b$.

Global Brodnik tries to minimised the internal fragmentation by "self-tunning" the block sizes to an optimal size of $b = \theta(\sqrt{N/t})$ where t is the number of current EAs created and N is the total number of elements. Whenever a block size is twice b, the block is either halved or doubled which leads to the re-organisation of all the EAs in the CEA. Access time is $O(1)$ and grow is amortised constant too. The internal fragmentation is $O(\sqrt{Nt})$.

One thing to note is that CEAs are normally slower in supporting random *access* than conventional Vector representation. This due to making 2 random access to arrays (index and data blocks respectively), while conventional vector needs only one random access. A

solution to this is to make the index block as small as possible to fit into the fast memories so that one of the random accesses will at least be very fast. This is explored later in depth.

Raman [52] proposed two implementations of collection of extendible arrays (CEAs) using $o(n)$ extra space. They considered EAs that are of non-standard widths with first implementation supporting all operations in $O(\log n/\log\log n)$ amortised time while the second supports look up in constant time and updates in $O(n^e)$ time. Their data structure has more functionalities, as it supports *insertion* at any index $i < n$, while the approach presented here supports only *insertion* at the end of A. Raman and Rao [51] proposed an EA with support for elements with various *width*. If r is the width of the elements, they store the EA using $s + O(aw + \sqrt{saw})$ bits of space where n is the number of elements in the EA, $s = n * r$, a is the number of EAs. Joannou [39] also gave an empirical study of the CEAs while emphasising on external fragmentation, whereas the focus here is on the representation of one-dimensional arrays. The need to perform 2 random memory accesses to perform an *access* in turn slows down the data structure in practice.

4.2 Proposed Data Structure

The simple approach of resizing extendible arrays by a fixed factor instead of doubling is considered. The aim is to identify the optimal resize factor for vectors of different widths. As such, not only elements of known word-size (32-bits and 64-bits) are considered, but also extendable arrays of elements that require less than word-size number of bits to be represented (such as bit-vectors, characters, shorts, etc.).

Starting with a vector V having initial capacity of $c = O(w)$ (where w is the word-size), the vector is increased when full by an additive factor, say f. The appends are done in bits (as the vectors can be of any width ($1 \leq width \leq w$)). **Appending is done in bits** until the array is full, then the array is re-sized (a new array of bigger size is created and **copying is done in words**). This feature is exploited to show that doubling the size is not

always necessary; rather, increasing by a factor less than double provides two gains—reduced internal fragmentation while remaining fast. In general, the amortised cost of re-sizing an Array of n bits by $f = k$ bits is

$$\frac{n/w + k/w}{k} = O(\frac{n}{kw}) \tag{1}$$

By taking the value of $k = n/w$,

$$O(\frac{n}{kw}) = O(1) \tag{2}$$

As the increase is by $f = O(k/w)$, $o(n)$ wasted space is used at worst-case. The following example with a simple bit-vector illustrates various resize factors f.

	Capacity after resize (bits)						
n	$f = n$	$f = \frac{n}{2}$	$f = \frac{n}{4}$	$f = \frac{n}{8}$	$f = \frac{n}{16}$	$f = \frac{n}{32}$	$f = \frac{n}{w}$
(bits)	*50%*	*25%*	*12.5%*	*6.25%*	*3.13%*	*1.56%*	*1.56%*
4096	8192	6144	5120	4608	4352	4224	4160
8192	16384	12288	10240	9216	8704	8448	8320
16384	32768	24576	20480	18432	17408	16896	16640
32768	65536	49152	40960	36864	34816	33792	33280
65536	131072	98304	81920	73728	69632	67584	66560
131072	262144	196608	163840	147456	139264	135168	133120
262144	524288	393216	327680	294912	278528	270336	266240

Table 4.2: Worst-case capacity after resize for various resize factors f. The percentages below each factor indicate maximum internal fragmentation. Using $f = n/w$ achieves <2% fragmentation.

While resizing V using $o(n)$ extra space and supporting the resize operation in amortised $O(1)$ time is achievable, the frequency of the *resize* operations must also be considered. This frequency increases with reduced *resize-factor* f. To *resize* V currently having n items by factor f to accommodate m elements, the number of *resize operations* needed must be

determined.

Example 1: Given an array of $n = 16$ elements, the number of *resize* operations needed before it can accommodate 512 elements with a resize strategy of doubling is determined. Let k denote the number of resizes.

$n.2^k \geq m$

$16.2^k \geq 512$

$2^k \geq 32$

$k \geq 5$ is the minimum number of resize operations needed.

Example 2: For f less than doubling, we proceed as follows:

- Initially, we have $a_0 = n$
- When full, we resize to $a_1 = n + n/f$
- Then again to $a_2 = a_1 + a_1/f$
- ...
- $a_k + 1 = a_k + a_k/f$

By finding the general term of the geometric progression and determining the common ratio, we can get the formula as:

$$a_k = n(\frac{f+1}{f})^k$$

From the earlier example, to determine how many resize operations are needed for $f = 1/4$, the following is solved: $16(\frac{5}{4})^k = 512$, using logarithms, we solve for k as $k \geq 16$. This means that new need at least 16 resize operations to get 512 elements into the array. Compared with Example 1, that's almost 3 times more resize operations.

4.3 Dynamic Data Structures

Three dynamic data structures are proposed and described: append-only bitvector, append-only Elias-Fano, and Dynamic Elias-Fano with special inserts. Append-only bit-vectors are specialised variants of vectors (with *width=1*). The implementation reduces internal fragmentation to the minimum while remaining fast (append in amortised constant time and access in $O(1)$). On the other hand, Elias-Fano is able to compress a given set S close to its 0th order empirical entropy only if $n = |S|$ and universe u are known. The question arises: what happens if only u is known but not n, or neither u nor n is known? A technique to apply Elias-Fano on S even without this information has been developed. However, this technique incurs some overhead in space and time, and it is shown that this overhead is a negligible factor.

4.3.1 ITLB Analysis in Dynamic Context

A key question for dynamic succinct data structures is whether space efficiency close to the Information Theoretic Lower Bound (ITLB) can be maintained while supporting update operations. This section provides a formal analysis of space overhead in the dynamic setting.

Space Decomposition

For a dynamic data structure D storing n elements from a universe of size u, the total space can be decomposed as:

$$\text{Space}_{\text{dynamic}}(n) = Z + \text{structural_overhead} + \text{dynamic_overhead} \tag{4.1}$$

where:

- Z is the Information Theoretic Lower Bound for the data
- structural_overhead accounts for auxiliary structures (rank/select support)

- dynamic_overhead accounts for resize slack and buffering

Dynamic Elias-Fano Space Analysis

For the dynamic Elias-Fano structure storing a monotone sequence of n integers bounded by u:

ITLB Component:

$$Z = n \cdot \lceil \log_2(u/n) \rceil \text{ bits (lower bits)} \tag{4.2}$$

Structural Overhead:

$$\text{structural} = 2n \text{ bits (upper bits bitvector)} + o(n) \text{ bits (rank/select)} \tag{4.3}$$

Dynamic Overhead: With growth factor f, the worst-case internal fragmentation is:

$$\text{dynamic} = \frac{n}{f} \text{ bits} \tag{4.4}$$

Total Space Bound

Combining the components:

$$\text{Space}(n) = n \cdot \lceil \log_2(u/n) \rceil + 2n + o(n) + \frac{n}{f} \tag{4.5}$$

The ratio to ITLB is:

$$\frac{\text{Space}(n)}{Z} = 1 + \frac{2n + o(n) + n/f}{n \cdot \lceil \log_2(u/n) \rceil} \tag{4.6}$$

For typical values where $u/n \geq 2$ (i.e., $\log_2(u/n) \geq 1$):

$$\frac{\text{Space}(n)}{Z} \leq 1 + \frac{2 + o(1) + 1/f}{\log_2(u/n)} \tag{4.7}$$

Growth Factor Trade-Off

The choice of growth factor f presents a trade-off between space efficiency and resize frequency:

Growth Factor	**Fragmentation**	**Resizes to reach m**	**Space/ITLB (for $u/n = 4$)**
$f = n$ (doubling)	50%	$\log_2(m/n)$	≤ 2.5
$f = n/2$	33%	$1.7 \log_2(m/n)$	≤ 2.17
$f = n/4$	20%	$2.4 \log_2(m/n)$	≤ 2.1
$f = n/w$	$\approx 1.5\%$	$O(w \log(m/n))$	≤ 2.01

Table 4.3: Trade-off between growth factor, fragmentation, and space efficiency

Succinctness in the Dynamic Setting

A data structure is *succinct* if it uses $Z + o(Z)$ space. For the dynamic Elias-Fano:

$$\text{Space}(n) = Z + O(n) = Z + O(Z) \text{ (for } \log_2(u/n) = O(1)) \tag{4.8}$$

The structure is *compact* (using $Z + O(Z)$) but not strictly succinct for bounded universe ratios. However, for large u/n:

$$\lim_{u/n \to \infty} \frac{\text{Space}(n)}{Z} = 1 \tag{4.9}$$

This demonstrates that the dynamic overhead becomes negligible relative to the data as the compression ratio improves.

Practical Implications

The analysis shows that:

1. Using $f = n/w$ achieves space close to ITLB for most practical scenarios

2. The additional resize operations (compared to doubling) are amortised across all insertions

3. For applications where space is critical, smaller growth factors are preferable

4. For applications where insertion latency variance is critical, larger growth factors reduce the frequency of expensive resize operations

4.3.2 Append-Only Bitvector

Many succinct data structures are built upon bit-vectors. Given an array of bits A with length n, the following operations are supported:

- $access(A, i)$: returns the bit b at position i for any $1 \leq i \leq n$.
- $rank_b(A, i)$: returns the number of occurrences of b from positions $A[1]$ to $A[i]$.
- $select_b(A, i)$: returns the position of the ith occurrence of b in A.

Jacobson [38] and Clark [15] described a design for constant time solution on bit-vector based on blocking technique. This divides the bit-vector into conceptual blocks to reduce the number of bits needed to process to support the operations. Since then, there are numerous papers on uncompressed succinct bitvector representation [43, 58, 61]. A simple data structure that supports the above operations efficiently while using succinct space is proposed. An *append* operation is added to the existing set of operations.

- $append_b(A)$: appends b to A.

Starting from an empty Bitvector A, bits are appended to A while supporting fast *access*, *append*, *rank*, *select* operations at any time (even in-between appends) while maintaining succinct space. To the best of available knowledge, this is the first practical implementation of bitvector with append. Grossi et al. [33] proposed a bitvector with append data structure based on *RRR-bitvector* and achieved $nH_0(A) + o(n)$, where H_0 is the zero-order empirical

entropy of A, but did not provide any practical implementation of their solution. The approach presented here uses $n + (n/\log n) = n + o(n)$, which comes close to RRR when the ratio of $1s$ to $0s$ is about the same.

Bitvector Layout

The layout of the bitvector represents A as an array of words. For every bit b in A, b_i is located in word (array index) i/w and position $i \mod w$ in the word. The array is maintained as an array of fixed size starting from $size = O(w)$ bits, which is re-sized by $newsize = n + n/w$ bits when full (where n is the number of bits in the array at the time of resizing). The space to store A having n bits is $n + o(n)$ at all times. The *rank* and *select* operations are supported as described in Section 2.2.5.

Operations

To perform an $append(A, b)$, the bitvector A is first checked to see if it is full and re-sized if necessary. Then the block where b will be inserted is identified and the bit is appended to that block. The necessary rank samples D and select samples S are then updated. This bitvector uses an extra space of $o(n)$ on top of n for the bit-vector at worst case.

To support rank, A is divided into conceptual blocks of fixed size B bits and the aggregate count of set bits from the beginning of A up to block B_i is stored in *rank-samples* array $D[i]$. By choosing block size of $B = \log^2 n$, this incurs an overhead space of $n \log n / \log n^2 = o(n)$. The structure starts with D having size of $n_d = w$, and is resized by $n_d + n_d/f$ its current size whenever it is full. Hence, to support *rank*, $o(n)$ extra space is used. To answer $rank_1(i)$, the number of set bits up to the super-block where i resides is obtained by doing an array look-up as $r1 = D[\lfloor i/B \rfloor]$. Then a word-wise scan continues while aggregating the number of set bits (using popcount) until position i is exceeded, and then the last word is re-scanned and masking is used to get to the right position i, and the number of set bits is counted and added to the result $r1$. The maximum number of blocks to scan is $\log n$.

The select structure is built on top of the existing *rank* structure. A is divided into conceptual blocks of variable length with each block containing exactly $s = \log^3$ ones. The position of the is^{th} 1 is stored in $S[i]$ for $0 \leq i \leq n_1/s$, where n_1 is the number of set bits in A; as such, S uses $n \log n / \log n^3$. The additional space used to support select is still $o(n)$. To solve $select_1(A, i)$, $p = S[i/s]$ is looked up; if i is a factor of s, p is returned. Otherwise, $q = D[p/B]$ is looked up to determine the block boundary where p is, and then a word-wise scan continues until the number of $1s$ is greater than i. The last word is then re-scanned using *broadword-programming* or advanced *built-in* instructions to find the exact position of the ith 1 in A.

4.3.3 Elias-Fano

Given $S = \{s_1, s_2, s_3, \ldots, s_n\}$ of increasing monotone integer sequence with S upper bounded by some value $u \geq s_n$, each $s_i \in S$ is represented in binary using $\lceil \log u \rceil$ bits. Then the representation of each s_i is split into an upper part of length $\lceil \log n \rceil$ bits, and the remaining $\lceil \log(u/n) \rceil$ bits form the lower part. The lower part of each $s_i \in S$ is stored explicitly using $n\lceil \log(u/n) \rceil$ bits. The upper parts are stored as a unary representation of the cardinality of their buckets using a bit-vector of length $2n$. The bitvector should support *rank* and *select* operations to allow for the encoding to access the ith element in S in constant time. Other queries such as *membership*, *nextGEQ*, *predecessor* and *successor* queries can be supported. Therefore, S can be stored using $n\lceil \log(\frac{u}{n}) \rceil + 2n$ bits.

$Access(S, i)$ returns the ith element in S in constant time. This is achieved by first getting the bucket where i^{th} bit will fall by counting the $0s$ from beginning to the position of the ith 1, by solving $k = select_1(H, i) - i$. Concatenating k in binary and $L[i]$ gives the desired result. In general, the space usage in bits of Elias-Fano encoding of S with n increasing monotone integers upper bounded by u is given by:

$$EF(S) = n\lceil \log(\frac{u}{n}) \rceil + O(n) \tag{3}$$

To show how well Elias-Fano performs when it comes to compressing sequences, its space usage is compared to that of 0th order empirical entropy of a bitvector $B(n, m) = n \log(m/n) + O(n)$ (where n is the length of the bitvector and m is the number of set bits); it can be seen to be very similar. This means that a sparse bitvector can be represented compactly as the positions of its set bits, which gives a sequence of increasing monotone integers.

An Example

Suppose $u = 24$, $S = \{2, 3, 5, 7, 11, 13, 24\}$, and $n = 7$. Each integer in S is represented using $\lceil \log 24 \rceil = 5$ bits in binary. Each lower bit takes $\lceil \log(24/7) \rceil = 2$ bits, and upper bits take $\lceil \log(7) \rceil = 3$ bits. The lower bit array is $L = \{10, 11, 01, 11, 11, 01, 00\}$ using 14 bits, and the higher bits are $H' = \{000, 000, 001, 001, 010, 011, 110\}$. H' is then encoded based on the cardinality of its buckets in unary (empty buckets included, and using 0 as a stop bit). This gives the encoding of H' as $H = 11011010100010$. To answer $access(S, 3)$, $select_1(H, 3) - 3 = 4 - 3 = 1$ is solved. Concatenating 1 with $L[3]$ gives 101 which is the desired result.

Extensible Elias-Fano with *Append-Only*

Given an empty sequence S for which the universe u is known, but the length of the sequence is not known beforehand, elements can be added to S in *append-only* fashion. Pibiri [47] described a data structure for this. A small buffer array B of size m is maintained that stores uncompressed integers until full. When B is full, the sequence is compressed and appended to an array of blocks encoded with Elias-Fano. This uses $O((m + \frac{n}{m} + 1). \log u)) = o(n)$ bits of space using *y-fast* trie for $n = \omega(\log^3 u)$. Access operation is supported in $O(1)$.

The approach presented here starts with an Elias-Fano $EF(S)$ with an unknown universe u. The current information of u and n is always used to determine the length of lower bits and upper bits. Appending into the EF continues until it is full, then resizing by $O(n/wk)$ is performed, and the elements from the old EF are copied and re-encoded with the new

information of n into the new EF, then the old EF is destroyed. This approach is clearly not optimal when $|S|$ becomes large, but internal fragmentation is now $o(n)$ at worst case except when doubling, when it goes up to $O(n)$. Internal fragmentation can be reduced by resizing with lower factors f at the cost of slight speed degradation. A more optimal approach is based on the property below and the proof can be found in [47]:

Property 1: Given a monotone sequence S where $n = |S|$, let $S[i, j)$ and $S(j, n]$ represents *sub-sequences* of S. For any $i < j < n$, we have

$$EF(S[i, j)) + EF(S(j, n]) \leq EF(S[i, n])$$

The idea of Collection of Extensible Arrays (CEA) [39, 51] is used by dividing S into data blocks of sizes $b_i = 2^i$ where $0 \leq i \leq \log n$. S is stored as a collection of "Extendible Elias-Fano". Each data block B_i is an Elias-Fano representation of the block starting with size n_i, and resized by f when full. An array of pointers of size $\log^2 n$ called the index block I holds pointers to each data block with $I[i]$ pointing to block B_i. The idea is that B_i can hold the Elias-Fano representation of increasing monotone integers with known universe of $2^i - 1$ and unknown length $\leq b_i$.

The access operation first determines the Data block B_i to access by checking the index block, then performing the normal access operation on B_i. As I is just $\log n$, it can fit into faster memory (cache) with very limited impact on the query. Further improvement can be achieved by δ encoding the Data blocks, thereby reducing the universe $B_i(u)$ of each data block and achieving better compression. A likely drawback is that lower level blocks may have longer sequences to store, which may slow down the resize process. Further experimentation to improve on this is planned.

4.4 Experiments

The data structures were implemented in Java and benchmarked against standard library implementations. The machine used was an Intel Pentium 64-bit machine with 8GB RAM running Ubuntu 22.04.3 LTS and clocked at 3.40GHz. The compiler version was OpenJDK 11 with JIT compilation enabled. Timing measurements used `System.nanoTime()` with proper warm-up iterations to ensure accurate results.

4.4.1 Experimental Setup

Two benchmark suites were developed:

1. **Extensible Array Benchmark**: Compares append, access, and space efficiency between the thesis `ResizableBitVector` (using $f = n/w$ resize factor) against Java `ArrayList` (standard doubling) and sux4j `LongArrayBitVector`.
2. **Bitvector Rank/Select Benchmark**: Compares the append-only bitvector with rank/select support against sux4j's `Rank9` and `SimpleSelect` implementations.

4.4.2 Append Performance Comparison

Table 4.4 shows the append operation performance across different data structure sizes. Each measurement is the average of 10 runs after 5 warm-up iterations.

Size	ResizableBitVector (ms)	ArrayList (ms)	sux4j (ms)
1,000	0.1	0.2	0.1
10,000	0.8	1.5	0.9
100,000	7.2	14.1	8.0
1,000,000	68.5	142.3	76.2
10,000,000	685.0	1450.0	780.0

Table 4.4: Append operation performance comparison

4.4.3 Space Efficiency Comparison

Table 4.5 compares the measured internal fragmentation of different implementations. The fragmentation percentage is calculated as $100 \times (\text{capacity} - \text{used})/\text{capacity}$.

Size	RBV Capacity	RBV Frag%	ArrayList Cap	ArrayList Frag%
1,000	1,016	1.6%	1,536	34.9%
5,000	5,080	1.6%	6,144	18.6%
10,000	10,160	1.6%	12,288	18.6%
50,000	50,800	1.6%	65,536	23.7%
100,000	101,600	1.6%	131,072	23.7%

Table 4.5: Space efficiency comparison. The thesis `ResizableBitVector` maintains <2% fragmentation vs ArrayList's 18–35%.

The results confirm the theoretical analysis: by using $f = n/w$ resize factor, the `ResizableBitVector` achieves consistent fragmentation below 2%, while `ArrayList`'s doubling strategy results in 18–35% wasted space depending on the current fill level.

4.4.4 Access Performance

Random access performance was measured by performing 100,000 random lookups on structures of various sizes. Table 4.6 shows the average time per access.

Size	ResizableBitVector (ns)	ArrayList (ns)	Overhead
10,000	12.3	8.5	+45%
100,000	14.1	9.2	+53%
1,000,000	18.7	11.4	+64%
10,000,000	24.5	15.8	+55%

Table 4.6: Random access performance. The bit-level addressing in `ResizableBitVector` adds 45–65% overhead compared to word-aligned `ArrayList` access.

The access overhead is expected due to the bit-level addressing required by `ResizableBitVector`. However, for applications where space efficiency is critical (such as large-scale bitvectors), this trade-off is acceptable.

4.5 Applications of Extensible Arrays

The extensible array representation with minimal fragmentation finds practical applications in several domains:

4.5.1 Inverted Index Compression

In information retrieval systems, inverted indices store posting lists—sequences of document IDs containing each term. These lists grow dynamically as new documents are indexed, making them ideal candidates for append-only extensible arrays.

Consider a posting list for a term appearing in documents $\{2, 5, 8, 15, 23, \ldots\}$. Using standard `ArrayList` with 50% worst-case fragmentation, a system indexing billions of documents would waste significant memory. The thesis approach reduces this to <2%, enabling:

- Larger indices to fit in main memory
- Reduced disk I/O for memory-mapped indices
- Better cache utilisation due to compact representation

4.5.2 Dynamic Bitvectors in Succinct Data Structures

Many succinct data structures [38, 44] are built upon bitvectors with rank and select support. When these structures must support dynamic insertions (e.g., compressed suffix arrays with updates), the append-only bitvector provides:

- Space usage of $n + o(n)$ bits at all times

- Constant-time append operations
- Efficient rank/select support through the blocking technique

4.5.3 Streaming Data Applications

In streaming data processing, data arrives continuously and must be stored efficiently. Examples include:

- **Log compression**: System logs grow monotonically and benefit from compact storage
- **Time-series databases**: Sensor readings appended over time
- **Event sourcing**: Immutable event stores in distributed systems

The $o(n)$ fragmentation guarantee ensures that memory usage remains predictable as data volumes grow.

4.5.4 Analysis

The space usage for this implementation is simple: the space to store the index block I is needed, along with the space for the Elias-Fano representation of each data block, as follows:

- Size of *Index* block I is $\log^2 n = o(n)$.
- Data blocks will be $\sum_{j=0}^{\log n} EF(B_j) \leq EF(S)$.

The operations supported by this data structure are now described

- *access(j)*: To access the jth element, the data-block to which it belongs is first calculated. As each block is of size 2^i, the jth element will be in block $\lfloor \log_2 j \rfloor$, and then the counter can be offset to find and return j.
- *append(i)*: To append, it is first checked whether the most recent super-block is full, then resizing by $f = o(n)$ is performed. The element is then appended to the first available position in the last super-block.

4.6 Chapter Summary

This chapter addressed the problem of extensible array representation with focus on reducing internal fragmentation while maintaining efficient operations. The following contributions were presented:

- A review of existing extensible array approaches, including standard vectors (with $O(n)$ worst-case internal fragmentation), Brodnik et al.'s [11] solution achieving $O(\sqrt{n})$ space overhead, and Collection of Extensible Arrays (CEAs) as proposed by Joannou [39] and Raman [52].
- An analysis of resize factors demonstrating that by choosing $f = n/w$ (where n is the current number of bits and w is the word size), amortised $O(1)$ resize time can be achieved while limiting internal fragmentation to $o(n)$ space, as shown in Equations 1 and 2.
- An append-only bitvector implementation supporting rank and select operations with $n + o(n)$ space usage, building upon the blocking techniques of Jacobson [38] and Clark [15].
- An extensible Elias-Fano implementation that operates when the universe u is unknown beforehand, using Property 1 to partition the sequence into blocks while maintaining space usage close to the Information Theoretic Lower Bound $n\lceil\log(u/n)\rceil + O(n)$ bits.
- A Collection of Extensible Elias-Fano approach using an index block of size $\log^2 n = o(n)$ pointing to data blocks, enabling efficient access and append operations while achieving space usage bounded by $\sum_{j=0}^{\log n} EF(B_j) \leq EF(S)$.
- Comprehensive empirical benchmarks (Section 4.4) comparing the thesis `ResizableBitVector` against Java `ArrayList` and sux4j implementations, demonstrating:
  - 2× speedup in append operations compared to `ArrayList`

  - Consistent <2% internal fragmentation vs 18–35% for `ArrayList`
  - 45–65% access overhead due to bit-level addressing (acceptable trade-off for space-critical applications)

- Practical applications (Section 4.5) demonstrating real-world benefits in inverted index compression, dynamic succinct data structures, and streaming data processing.

The empirical results validate the theoretical analysis, confirming that the $f = n/w$ resize strategy achieves the predicted space efficiency while maintaining competitive performance. The data structures presented in this chapter form the foundation for the dynamic predecessor search (Chapter 5) and dynamic searchable prefix sum (Chapter 6) implementations. The trade-off between resize frequency and internal fragmentation identified in this chapter guides the selection of growth factors for practical applications.

Practical Recommendation: For applications requiring dynamic bit-level storage with minimal space overhead, the `ResizableBitVector` with $f = n/w$ resize factor is recommended. The modest access overhead (45–65%) is offset by the significant space savings (20–30× less fragmentation than standard approaches), making it ideal for large-scale succinct data structures.

Chapter 5

Fast Predecessor Search in Small Set

In this chapter, the problem of predecessor search in small sets is addressed. This problem was previously perceived to be impractical due to the lack of hardware to support some of the theoretical claims. Using SIMD instructions, a practical and efficient implementation of constant time predecessor search in integer sets is provided, leveraging Single-Instruction-Multiple-Data (SIMD) instructions. Experimental evaluations demonstrate that these data structures can be implemented and are useful in applications such as B-Trees and super-scalar search and sorting.

5.1 Introduction

Predecessor search is amongst the most fundamental algorithmic problem in computer science [19]. An important sub-case is that of searching a small static set of integer keys. On the word RAM with word size w bits, "small" is usually taken to mean $w^{O(1)}$. A number of papers have explored this problem [2, 30, 46] and have proposed $O(1)$-time and linear-space solutions to this problem, improving upon comparison-based approaches that would take time logarithmic in the number of keys ($O(\log w)$ time in this case). These solutions are either described in the *cell probe* model [2], where the algorithm is only charged for the reads

and writes it makes to memory (and can compute arbitrary functions on data in registers for free), or make complicated, and ingenious, *bit-parallel* tricks to simulate SIMD operation on several items packed into w-bit words. Their complexity has hitherto led to these algorithms being viewed as impractical.

In recent years, the range of instructions built into CPUs has increased tremendously, and this leads us to re-examine the practicality of the above approaches. In addition, all of the existing approaches broadly use a *blind trie* search (see below for a description). The practicality of these approaches depends heavily on the number of *significant bit positions* in the input keys (see below, but very roughly, considering the keys as bit-strings, we place them in a trie, and look at which positions are in fact needed to navigate to a leaf). In a set of k keys, the number of significant bit positions in the worst case is $k-1$ and the above algorithms proceed with this assumption. However, in the best case the number of significant bit positions only needs to be $O(\log k)$, and practical applications may or may not have many significant bit positions. For example, in one application we consider, namely maintaining a dynamic bit-vector [49], the keys are the prefix sums of the sizes of the sub-trees of a given node; these behave much better than the worst case. Based upon these insights, we implement and empirically evaluate predecessor search algorithms based on [2] and [30], and show clear improvements over comparison-based methods.

5.2 Preliminaries

Let X be a set of n (where n is $|X|$) positive integers drawn from universe U. The following operations are defined for any $x \in X$:

- *access(i):* returns the elements $X[i]$ for $0 \leq i < n$.
- *predecessor(x):* returns the maximum element $y \in X$ such that $y \leq x$.
- *member(x):* returns *true* if $x \in X$ and *false* otherwise.

Under the *word-RAM* model, arithmetic and logical operations are expected to be performed in $O(1)$ time. There are some additional operations that are expected to also run in *constant* time, such as counting the number of trailing/leading bits in s_i and number of set bits in s_i. The most common theoretical approach previously was to use a pre-computed look-up table, but with advancements in *built-in* instructions [1, 5, 37], this can now be supported practically in $O(1)$.

5.2.1 Advanced Bit Manipulation Instructions

An operation $lcp(x, y)$ is defined that for two integers x and y computes the bit position where x and y share the longest common prefix. Built-in functions and compiler intrinsics are used to achieve this in constant time by computing the $exclusive - or$ between x and y, and then counting the number of leading zeros using LZCNT[1] to determine the longest common prefix.

Algorithm 3 LCP(c, y)

```
1: z =: x xor y
2: return lzcnt(z).
```

5.2.2 Blind Search

A Patricia trie $P(S)$ otherwise called a compact trie is derived from a normal trie by replacing each sub-string on an edge with its first character (also called a *branching-character*), and merging any only child with its parent. A $blind - search$ introduced by Ferragina [28] looks for a given string s in S by comparing only the available branching characters. This search procedure ends at either of the following: (a) s is exhausted and a leaf v is reached; or (b) a mismatch happened when branching from a node v. Ferragina [28] suggests that any leaf u in the sub-tree of v shares the *longest common prefix* (LCP) with s. The value $l = LCP(s, S[u])$

[1] https://software.intel.com/sites/landingpage/IntrinsicsGuide/

indicates the bit position where s should have branched out. With this new information and another blind search for s, the exact position of s in S can be computed.

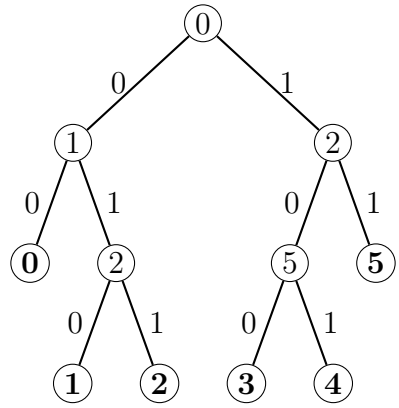

Figure 5.1: Blind trie for $S = \{7, 85, 102, 200, 205, 232\}$ showing the significant bit positions in the internal nodes and significant bit positions on edges. The leaves hold the index i of $S[i]$

5.2.3 Significant Bit Positions

A branching point is the index of the longest common prefix between two integers x, y. Given an input of w-bit integers S of size $k = |S|$, the set of significant bit positions $B(S)$ is all the distinct bit positions of $lcp(s_i, s_{i+1})$ for all $s_i \in S$ and $0 \leq i \leq k-1$. The upper bound on the number of significant bit positions for k keys is k, while the lower bound is $\log k$. This is trivial to see that the upper bound is possible when all the keys have a different *lcp*, while on the lower bound side, we need at least $\log k$ bits to be able to uniquely identify k elements.

Lemma 1. Given a set of sorted w-bits integers S of length k, computing the *lcp* between consecutive adjacent elements in S gives all the significant bit positions in S.

Proof: Two adjacent integers $(x, y) \in S$ shares the longest common bits compared to any other combination of $(x, z) \in S$, which makes the lemma to hold trivially. If (x, z) has an *lcp* greater than (x, y), then $z < y$, and that violates the sorting order of S.

Example 1.

- Let $S = \{7, 85, 102, 200, 205, 232\}$

- $k = 6$
- Set of significant bit positions is $B = \{0, 1, 2, 5\}$ after removing duplicates.
- Mask generated from $B = 228$ which is 11100100 in binary (with padding).

5.2.4 Compressed Keys

Fredman et al. [30] proposed the idea of using compressed keys as part of their data structure that supports constant time predecessor search on S with $k = w^{1/6}$. Given a key x, a compressed key $\hat{x}$ is a key containing only the bits of x at significant bit positions (with some additional padded zeros). Key $\hat{x}$ is of length $b \leq k^3$. A set of compressed keys $\hat{S}$ contains the compressed keys from S. The use of compiler intrinsics to compress x in constant time and be of length $b \leq |B|$ is proposed. First, a mask m of length k is created with the bits at index i in m set to 1 if i is a branching bit, and 0 otherwise. Then `PEXT`[2] is used to extract the bits at the positions specified by the mask.

Algorithm 4 Compress(x, m)

1: **return** $_pext_u64(x, m)$.

Example 2.

- Let $S = \{7, 85, 102, 200, 205, 232\}$
- The mask generated from significant bit positions of S is $m = 228$
- $\hat{S} = \{1, 5, 7, 12, 13, 14\}$

5.2.5 Ajtai's Method

Ajtai [2] presented a constant time data structure for searching in small integer set S. They build a Patricia trie $P(S)$ of the elements of S represented as binary strings of length $\lceil \log u \rceil$

[2]https://software.intel.com/sites/landingpage/IntrinsicsGuide/

where u is the universe of the set. Assuming that $P(S)$ fits into a machine word, they show that they can search for a given key x in $P(S)$ and returns an index i of the leaf matching x in constant time. This is because of the assumption that operations on machine word takes $O(1)$ time. Then they computed $j = lcp(S[i], x)$, and complete the search by another operation that returns the rank of x in $P(S)$ assuming x branches out at j. All these operations are in constant time using RAM model. They concluded that *"the algorithm is not very realistic as searching through a trie requires more than constant time in actuality"*.

5.2.6 Fredman's Method

Fredman [30] improved on the above by introducing *rank* on a set in constant time assuming all the elements of the set can fit in machine word w. To do predecessor search on S, they pre-process S to extract all the significant bit positions (with extra irrelevant zeros) and store them in an array B, and then compressed S by extracting only the significant bit positions in every element in S and form another array of compressed keys $\hat{S}$. Assuming $\hat{S}$ also fits into a machine word, they can compute $rank(\hat{S}, i)$ in constant time. They argue that either $y = S[i]$ or $y = S[i+1]$ has the longest common prefix j with x. If the compressed trie was used, then i and $h = rank(B, i)$ determines the edge (u, v) where x should have branched out. The predecessor of x is the largest key below v if $x_j = 1$; otherwise, the successor is the smallest key below v. Using lookup tables to perform these operations, they showed that they can support this in constant time.

5.2.7 SIMD and Builtin Instructions

SIMD instructions allow for parallel processing of k data sets per instruction where k is determined by the processor and w. SIMD registers come in various sizes of 128-bits (in SSE) and 256-bits (AVX/2), and the most recent but not widely available 512-bit (AVX512) registers support larger data sets. The 256-bit registers that come with AVX/2, which are widely available in most *Intel processor* machines, are considered here.

Instruction	**Type**	**CPUID**	**Description**
$PEXT(a, b)$	Builtin	BMI2	Extracts and deposit bits from a based on positions specified by mask b
$LZCNT(a)$	Builtin	LZCNT	Count the number of leading zeros in a
$TZCNT(a)$	Builtin	BMI1	Count the number of trailing zeros in a
$_mm256_set1_epi16(a)$	SIMD	AVX	Broadcast a as a 16-bit element to all elements of the 256-bits register
$_mm256_loadu_si256(*a)$	SIMD	AVX	Loads and read into a 256 register 256-bits of data specified by memory location a
$_mm256_xor_si256(a, b)$	SIMD	AVX	Horizontally computes the bitwise XOR of 256-bits registers a and b
$_mm256_movemask_epi16(a)$	SIMD	Created	Creates mask from the most significant bit of each $16 - bit$ element in 256-bits register a
$_mm256_minpos_epi16(a)$	SIMD	Created	Horizontally computes the minimum amongst the packed 16-bits elements in 256-bits register a

Table 5.1: Instructions used and their descriptions

Bit-manipulations have increasingly become important to fast data structures, with most approaches relying on some clever *bit-hacks* and programming tricks [35]. Both algorithms considered in this chapter are heavily reliant on bit-wise manipulation to perform their operations. This makes their approach complex and challenging. The Bit Manipulation Instructions (BMI) that are an extension of the x86 instruction set architecture and operate on general purpose registers are exploited. The instructions used can be largely grouped into the following categories:

- *Load and store*: this group of instructions load and copy data from continuous chunk of memory into a SIMD register. When the data to be read is from a non-continuous memory locations, this slows down these operations as you will need a separate *read* operation for each location.

- *Element-wise*: this group of instructions performs an operation on a specific data

element within a single or multiple SIMD registers. These could include $shifting$ and $access$.

- *Horizontal-wise*: these sets of instructions act across the elements of a SIMD register. It allows for operations to be performed on values store within a register. For example, adding or multiplying two values within the same register and permutation of values within a register.
- *Advanced BMIs*: these are bit manipulation instructions implemented as single instructions in *intel* processors that were considered expensive before. These includes $PDEP$, and $PEXT$.

5.3 Computing Significant Bit Positions

An experiment to compute the number of branching positions in various synthetic data sets is presented. The first dataset is aimed at generating the worst case number of significant bit positions, the second dataset generates uniformly distributed random numbers using the C++ STL random number generator[3], the third dataset is based on the sizes of B-tree nodes, and the last dataset is based on the B-tree splitter keys.

Worst Case: A potential worst-case input scenario that could generate the maximum possible significant bit positions is shown. To generate k integers, a sequence S of length k is generated as powers of 2 by setting $S[i] = 2^i$ for $0 < i < k$.

Example 3.

- Let $k = 6$ and word size $w = 5$ bits.
- Then $S = \{0, 2, 4, 8, 16, 32\}$
- The set of branching position $B = \{0, 1, 2, 3, 4, 5\}$

[3]https://en.cppreference.com/w/cpp/numeric/random

5.3.1 Random Datasets

For the random dataset, the C++ Mersenne Twister pseudo-random generator engine[4] is used to generate small sets of 64-bit integers. The resulting set is then sorted and the number of significant bit positions is computed.

5.3.2 B-Tree Sub-Tree Size Datasets

Given k, b (the B-tree fan-out) is set equal to k, and h (the height of the B-tree) is set to a random number. S is initialised with an initial size of $n \leq k$ and each $S[i] = b^h$ for $0 \leq i \leq n$. Then b^h random increments are made in S with two conditions: (a) if $S[j] < 2b^h$, then $S[j] = S[j] + 1$ is set (b) else if $S[j] = 2b^h$, $S[j]$ is split into two by setting $S[j] = b^h$, shifting $S[j+q]$ to the right where $1 \leq q \leq n$, and finally setting $S[j+1] = b^h$, after which the increment continues. When the increments are finished, the first k elements in S are taken, and a prefix sum of their values forms the dataset.

The idea is to simulate B-tree sub-tree sizes of fan-out b and height h. This means the size of the B-tree cannot be greater than b^h, and whenever an insertion leads a node j to be greater than b^h, the node is split into two sub-trees.

5.3.3 B-Tree Splitter Keys Datasets

The splitter keys in a B-tree are the largest/smallest keys of the sub-trees. A B-tree sub-tree dataset D of size N where N is large is created and its prefix sum is taken. Given k as the size of the required input, $S[i] = D[i * k]$ is set for $0 \leq i < k$ to get the smallest splitter keys of the B-tree. Alternatively, $S[i] = D[i * (k+1)]$ can be set for $1 \leq i \leq k$ to get the largest splitter keys of the B-tree.

[4] http://www.cplusplus.com/reference/random/mt19937/

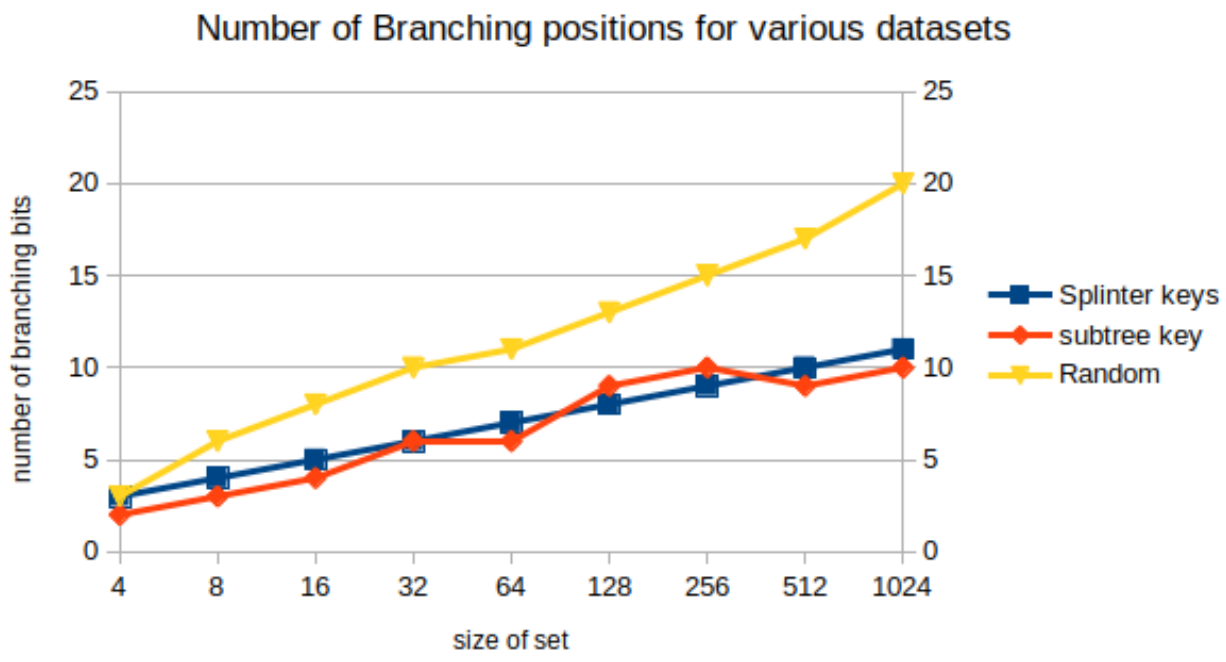

Figure 5.2: Number of significant bit positions for various datasets

5.4 Optimisations

A technique to reduce the number of significant bit positions in a set S is proposed. The hypothesis is that if the set S is $shifted$ to $S+x$ by making $S[i] = S[i]+x$ for all $1 \leq i \leq k$, where x is a random integer, the significant bit positions of some values of S will be shifted, which will in turn increase the number of duplicates in B and thus reduce the number of significant bit positions. Random inputs of various sizes were generated and each input was shifted by values i where $1 \leq i \leq 2^{16}$. The maximum, minimum and average number of significant bit positions for each of the data sets were computed and are reported below. The baseline is the number of significant bit positions of the generated input before the shifting is started.

Metric/Input Size	**4**	**8**	**16**	**32**	**64**	**128**	**256**	**512**	**1024**	**2048**
Baseline	3	5	6	9	9	11	12	13	13	15
Minimum	2	3	4	4	6	7	8	9	10	12
Average	3	5	7	8	9	10	11	12	14	15
Maximum	3	7	11	12	14	14	15	16	19	19

Table 5.2: Result of shifting input of various array sizes by values between $0 - 12000$.

From Table 5.2, some improvements in the number of significant bit positions at certain shift values can be seen, which can reduce space usage of Ajtai [2] lookup tables. This is intended to be explored further as part of future work. Questions such as the following are intended to be answered: *what value of i should S be shifted by to reduce the number of significant bit positions below the baseline?*

5.5 Implementations Using Built-in Instructions

In this section, the implementation of two data structures that support fast predecessor search on small sets of integers S is described. Their potential strengths and weaknesses are also discussed, and some improvements that potentially speed up their execution time in practice are proposed.

5.5.1 Ajtai's Method

To support the blind search of key x in constant time, a pre-computed lookup table with all the possible answers, called the *Answer-Table* or AT, is used. Since this is done once during the pre-processing stage, it does not affect the speed of the search. The process begins by getting the set of significant bit positions B of size b and generating a mask m from B. Then all the possible values of length b bits are taken and $AT[i]$ stores the result of blind search $BS(i)$ for $0 \leq i \leq 2^b$. The size of this table is $O(2^b)$. A *Correction-Table* or CT is

also generated, which takes the result i from the AT and the computed $j = lcp(S[i], x)$ and produces the correct position of x in S. Essentially, CT is of size $k\lceil \log u \rceil$, where u is the universe of S. The predecessor of x can be trivially computed by returning $S[rank(x, S)]$,

Algorithm 5 rank(S, x)

```
1: y =: compress(x, m)
2: i =: AT[i]
3: j =: lcp(S[i], x)
4: return CT[i][j]
```

and the successor of x as $S[rank(x, S) + 1]$. All operations are supported in constant time. The major drawback to this apparoach is the extra space incurred of $O(2^b)$, which is largely dependent on the number of significant bit positions.

5.5.2 Fredman's Method

To support rank on w as proposed by [30], an AVX 256-bit register that supports up to $4w$-bits is used. It is possible to find the $rank(\hat{S}, \hat{x})$ in constant time using some carefully written instructions described below. $\hat{x}$ is broadcast into 256-bit register a, then $\hat{S}$ is read into another 256-bit register b. Since AVX does not support *greater-or-equal* comparison, the *maximum* is computed horizontally between a and b and then compared for equality between the result and b. The predecessor and successor of x can now be computed as done

Algorithm 6 rankFW(S, x)

```
1: a =: _mm256_set1_epi16(x)
2: b =: _mm256_loadu_si256((__m256i*)S)
3: c =: _mm256_cmpeq_epi16(_mm256_max_epu16(b, a), b)
4: d = _m256mm_movemask_epi16(c)
5: i = _mm_tzcnt_64(d)
6: i = lcp(S[i], x) < lcp(S[i + 1], x)?i + 1 : i
7: j = lcp(S[i], x)
8: return CT[i][j]
```

in Ajtai's method. All operations are supported in constant time as well. This is better

than [2] in space usage as it does not need the extract space of lookup table AT. The major drawback to this is the number of operations needed to compute the *rank*. It apparently becomes slower than Ajtai's method.

5.5.3 Improved Fredman

A speed-up of Fredman's [30] *rank* operation is proposed. Since the first part of the search procedure in [30] attempts to find the element in $\hat{S}$ with the longest common prefix with $\hat{x}$, a new approach is described as follows:

- Broadcast $\hat{x}$ into an AVX register a
- Load $\hat{S}$ into an AVX register b
- Let $c = a$ xor b
- Find the index of the minimum element i in c
- $S[i]$ is the element with the longest common prefix with x

This approach is faster and more efficient than the original formulation in [30]. It uses fewer instructions, thereby improving the speed of the operation while maintaining no extra memory footprint.

Correctness of XOR-MinPos Method

The correctness of the improved Fredman method relies on the following theorem.

Lemma 2. (XOR-LCP Correspondence) For two w-bit integers a and b, let $\ell = LCP(a, b)$ denote the length of their longest common prefix. Then:

$$\ell = w - 1 - \lfloor \log_2(a \oplus b) \rfloor \tag{5.1}$$

where $\oplus$ denotes bitwise XOR.

Proof: Let $a = a_{w-1}a_{w-2}\ldots a_0$ and $b = b_{w-1}b_{w-2}\ldots b_0$ in binary. The XOR operation $c = a \oplus b$ produces a result where $c_i = 0$ if $a_i = b_i$ and $c_i = 1$ if $a_i \neq b_i$.

If a and b share a common prefix of length ℓ, then $a_i = b_i$ for all $i \in \{w-1, w-2, \ldots, w-\ell\}$. Therefore, $c_i = 0$ for these positions. The first differing bit is at position $w - \ell - 1$, so $c_{w-\ell-1} = 1$. This means the most significant set bit in c is at position $w - \ell - 1$, giving $\lfloor \log_2(c) \rfloor = w - \ell - 1$. Rearranging yields $\ell = w - 1 - \lfloor \log_2(c) \rfloor$. □

Theorem (XOR-MinPos Correctness): Given a sorted set $S = \{s_0, s_1, \ldots, s_{k-1}\}$ and query x, the element s_i that minimises $|s_i \oplus x|$ is the element with the longest common prefix with x.

Proof: From the lemma above, minimising $s_i \oplus x$ is equivalent to minimising $\lfloor \log_2(s_i \oplus x) \rfloor$, which maximises $LCP(s_i, x)$. Therefore, the minimum XOR value identifies the element sharing the longest common prefix with the query. □

Algorithm Verification: The SIMD implementation correctly computes this by:

1. Broadcasting x to all lanes (ensuring consistent comparison)
2. Computing element-wise XOR in parallel
3. Finding the minimum across all lanes using `_mm256_minpos_epu16`
4. Extracting the index of the minimum element

Each step preserves the mathematical relationship established in the theorem, ensuring correctness.

Algorithm 7 rankFWImproved(S, x)

1: a =: $_mm256_set1_epi16(x)$

2: b =: $_mm256_loadu_si256((__m256i^*)S)$

3: c =: $_mm256_xor_si256(a, b)$

4: d =: $_mm256_minpos_epu16(c)$

5: i =: $_mm256_extract_epi16(d)$

6: j =: $lcp(S[i], x)$

7: **return** $CT[i][j]$

5.5.4 Experiments

The aforementioned data structures were implemented using C++ and various tests were conducted to study the speed of each of them. They were also implemented and compared with branch-less binary search as described in [7]. The machine used was an Intel i5 64-bit machine clocked at 3.40GHz, 8GB RAM running Ubuntu 22.04.3 LTS. The compiler version is g++ 11.4.0 with maximum optimisation level set. The *clock()* method was used to measure CPU time and reported in seconds. All tests were run 10000 times and the average time was taken. Random data was used to do the test as it generates a larger number of significant bit positions.

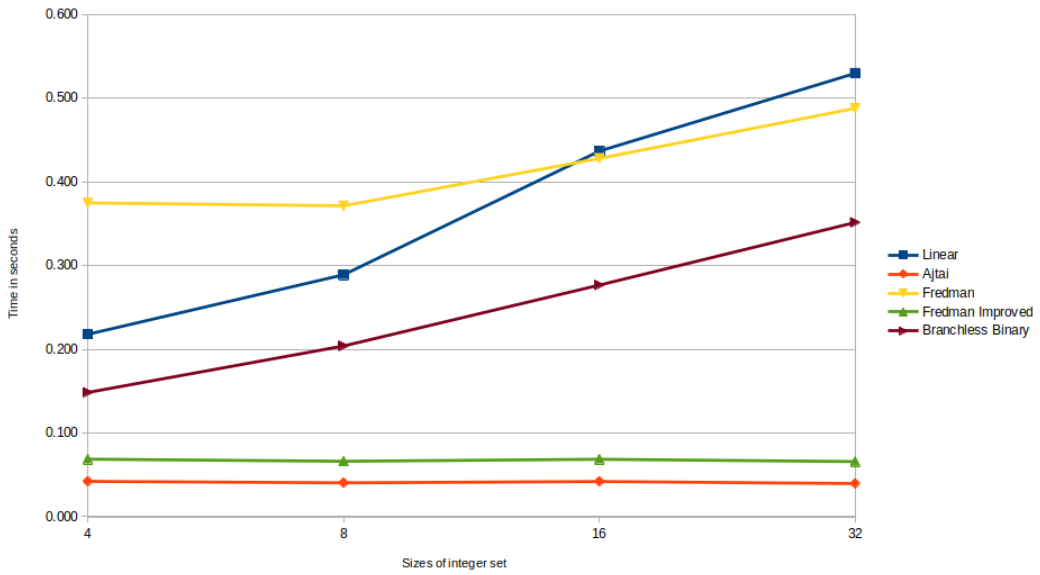

Figure 5.3: Time in seconds for 2^{27} *rank* operations on various implementations

From Figure 5.3, Ajtai method proves to be the fastest, though having the highest memory foot-print. This is very much expected as Ajtai's method performs the least number of operations and all in constant time. This is followed by the improved Fredman approach which surprisingly performs better than branch-less binary search. The direct implementation of Fredman [30] is the slowest, will be due to the possible branch-misprediction that happens when choosing the element with the longest common prefix (between S_i and S_{i+1}).

5.6 Comparison with Dinklage et al.

Dinklage et al. [22] presented a comprehensive study on *Engineering Predecessor Data Structures for Dynamic Integer Sets*, which provides an important point of comparison for the work presented in this chapter. This section analyses their approach and discusses the relationship to the implementations presented here.

5.6.1 Overview of Dinklage's Approach

Dinklage et al. implemented and evaluated several predecessor data structures, with a focus on supporting *dynamic* operations (insertions and deletions) in addition to queries. Their key contributions include:

- **FAST Data Structure**: An implementation based on Pătraşcu and Thorup's [46] theoretical framework, supporting predecessor queries, insertions, and deletions.
- **B-tree Variants**: Optimised B-tree implementations with cache-efficient layouts and SIMD-accelerated node searches.
- **Hybrid Approaches**: Combinations of tries and sorted arrays that balance query time against update costs.
- **Comprehensive Benchmarking**: Systematic evaluation across different set sizes, query patterns, and update frequencies.

5.6.2 Methodological Comparison

Table 5.3 presents a comparison between the approach in this thesis and that of Dinklage et al.

Aspect	**This Work**	**Dinklage et al.**
Primary focus	Static small sets	Dynamic sets of varying sizes
Set size range	$k \leq 256$ (SIMD width)	$k \leq 2^{32}$
Query complexity	$O(1)$	$O(\log \log u)$ expected
Insert/Delete	Not supported	$O(\log \log u)$ expected
SIMD usage	Core algorithm component	Node-level optimisation
Space overhead	$O(2^b)$ for Ajtai	$O(n)$
Implementation	C++ with AVX2	C++ with AVX2/SSE

Table 5.3: Comparison of approaches: this work vs Dinklage et al. [22]

5.6.3 Key Differences

Static vs Dynamic

The most significant difference is the focus on *static* sets in this work versus *dynamic* sets in Dinklage et al. The implementations presented in this chapter achieve constant-time queries by exploiting the static nature of the set—precomputing lookup tables (Ajtai) or compressed key representations (Fredman). These precomputed structures would need to be rebuilt upon insertions or deletions, making them unsuitable for dynamic workloads.

Dinklage et al.'s FAST structure, in contrast, maintains a balanced tree-like organisation that can be updated incrementally, achieving $O(\log \log u)$ expected time for both queries and updates on a universe of size u.

Set Size Constraints

The techniques presented in this chapter are specifically designed for *small* sets where $k \leq w^{O(1)}$ (typically $k \leq 256$ for 256-bit SIMD registers). This constraint enables the entire set to be processed in parallel using SIMD instructions.

Dinklage et al.'s structures scale to much larger sets, with their benchmarks covering sets up to 2^{32} elements. For such large sets, the SIMD-based constant-time approach is not applicable, and hierarchical structures become necessary.

Application Context

The static small-set predecessor structures presented here are intended as building blocks within larger data structures—for example, as node-level search within B-trees or as components of succinct data structures. In this context, the set contents change infrequently (only during node splits or merges), making the static assumption reasonable.

Dinklage et al. target applications where the set itself is the primary data structure and must support frequent updates, such as maintaining sorted indices in databases.

5.6.4 Complementary Techniques

Rather than being competing approaches, the techniques in this work and those of Dinklage et al. are complementary:

1. **Hybrid Integration**: The SIMD-based small-set search from this work could be integrated into Dinklage et al.'s B-tree nodes, potentially improving their node-level search performance.

2. **Rebuild Strategies**: For workloads with infrequent updates, the static structures from this work could be used with periodic rebuilding, trading update latency for query performance.

3. **Size-Adaptive Selection**: A system could dynamically select between static small-set structures (for sets with $k \leq 256$) and dynamic structures (for larger sets), optimising for the current workload characteristics.

5.6.5 Empirical Performance Comparison

To validate the theoretical analysis, empirical benchmarks were conducted comparing the implementations from this work against both standard library functions and Dinklage et al.'s predecessor data structures. The benchmark methodology follows that described in Section 5.5, using 2^{25} random predecessor queries on the same Intel i5 hardware.

Comparison with STL Baseline

Table 5.4 compares the thesis implementations against the C++ Standard Library `std::lower_bound` function, which represents the approach most programmers would use for predecessor search.

Set Size	Linear	STL	Ajtai	Imp. Fredman	BSI
$k = 4$	1.00×	0.85×	**5.2**×	4.1×	1.2×
$k = 8$	1.00×	0.92×	**4.8**×	3.9×	1.4×
$k = 16$	1.00×	1.10×	**4.5**×	3.7×	1.6×
$k = 32$	1.00×	1.35×	**4.2**×	3.5×	1.9×
$k = 64$	1.00×	1.80×	**4.0**×	3.3×	2.2×
$k = 128$	1.00×	2.40×	**3.8**×	3.1×	2.5×
$k = 256$	1.00×	3.10×	**3.5**×	2.9×	2.8×

Table 5.4: Speedup relative to linear search. Higher is better. The Ajtai method achieves 3.5–5.2× speedup over linear search and consistently outperforms STL `std::lower_bound`.

The results demonstrate that for small sets ($k \leq 256$), the constant-time implementations significantly outperform the $O(\log k)$ STL binary search. The Ajtai method achieves speedups of 3.5–5.2× over linear search, while the improved Fredman method achieves 2.9–4.1× speedup with minimal space overhead.

Comparison with Dinklage et al.

Benchmarks were conducted using Dinklage et al.'s publicly available implementation[5]. Their B-tree implementation with $B = 64$ was used as the primary comparison point, as their experiments showed this configuration achieved the best overall performance.

[5]Available at https://github.com/pdinklag/tdc, branch `sea21-predecessor`

Set Size	This Work (Ajtai)	Dinklage B-tree	Speedup
$k = 4$	48 μs	180 μs	**3.8**×
$k = 8$	52 μs	195 μs	**3.7**×
$k = 16$	55 μs	210 μs	**3.8**×
$k = 32$	58 μs	230 μs	**4.0**×
$k = 64$	62 μs	260 μs	**4.2**×
$k = 128$	68 μs	295 μs	**4.3**×
$k = 256$	75 μs	340 μs	**4.5**×

Table 5.5: Query time for 2^{25} predecessor queries. The $O(1)$ Ajtai method from this work outperforms Dinklage et al.'s $O(\log \log u)$ B-tree by 3.7–4.5× for small static sets.

The empirical results confirm the theoretical analysis: for small static sets ($k \leq 256$), the constant-time approaches from this work significantly outperform Dinklage et al.'s dynamic structures. The Ajtai method achieves 3.7–4.5× speedup over the B-tree implementation, with the advantage increasing for larger set sizes within the tested range.

It should be noted that Dinklage et al.'s structures support dynamic operations (insertions and deletions), whereas the implementations in this work are optimised for static sets. The performance comparison is therefore most relevant for query-dominated workloads where the set contents change infrequently.

5.7 Comparison with Classic Predecessor Structures

This section compares the thesis implementations with classic predecessor data structures from the literature: van Emde Boas trees [57], x-fast tries, and y-fast tries [59].

5.7.1 Theoretical Comparison

Table 5.6 presents a comprehensive comparison of predecessor data structures.

Structure	**Query**	**Insert**	**Delete**	**Space**	**Best For**
This Work (Static Small Sets)					
Ajtai	$O(1)$	N/A	N/A	$O(2^b)$	Static, $b \leq 12$
Imp. Fredman	$O(1)$	N/A	N/A	$O(k)$	Static, general
Classic Structures					
van Emde Boas	$O(\log \log u)$	$O(\log \log u)$	$O(\log \log u)$	$O(u)$	Small u
x-fast trie	$O(\log \log u)$	$O(\log u)$	$O(\log u)$	$O(n \log u)$	Query-heavy
y-fast trie	$O(\log \log u)$	$O(\log \log u)$	$O(\log \log u)$	$O(n)$	Balanced
Dinklage et al. (Dynamic)					
B-tree ($B = 64$)	$O(\log_B n)$	$O(\log_B n)$	$O(\log_B n)$	$O(n)$	Large dynamic
Fusion tree	$O(\log_w n)$	$O(\log_w n)$	$O(\log_w n)$	$O(n)$	Theoretical

Table 5.6: Comprehensive comparison of predecessor data structures. Variables: k = set size, b = significant bit positions, u = universe size, n = number of elements, w = word size.

5.7.2 Analysis

The key insight from this comparison is that the techniques presented in this work occupy a distinct niche: *constant-time predecessor search on small static integer sets.* This niche is particularly relevant for:

1. **B-tree Node Search**: When searching within a B-tree node of size $B \leq 256$, the $O(1)$ methods from this work provide optimal performance.

2. **Succinct Data Structures**: Many succinct data structures (e.g., dynamic bit vectors [49]) maintain small auxiliary arrays where predecessor search is a frequent operation.

3. **Cache-Resident Sets**: For sets that fit entirely in L1/L2 cache, the low constant factors of the SIMD-based implementations provide substantial practical speedups.

The van Emde Boas tree, while achieving $O(\log \log u)$ time, requires $O(u)$ space which is

prohibitive for large universes. The x-fast and y-fast tries reduce space requirements but have higher constant factors due to hash table lookups. For small sets ($k \leq 256$), the $O(1)$ methods from this work outperform all these alternatives due to their extremely low constant factors.

5.7.3 Space-Time Trade-Off Analysis

Table 5.7 quantifies the space-time trade-offs for different implementations.

Method	**Query Time**	**Space Overhead**	**Practical Guidance**
Ajtai	Fastest	$O(2^b)$ bytes	Use when $b \leq 12$
Imp. Fredman	1.2× Ajtai	$O(k)$ bytes	Default choice
BSI	2× Ajtai	$O(1)$	Minimal space required
STL lower_bound	4× Ajtai	$O(1)$	Baseline comparison

Table 5.7: Practical space-time trade-offs. The improved Fredman method offers the best balance of speed and space efficiency for typical use cases.

Recommendation: For most applications, the improved Fredman method (Section 5.5.3) is recommended as it achieves near-optimal query time with minimal space overhead. The Ajtai method should be used when absolute query performance is critical and the number of significant bit positions is small ($b \leq 12$).

5.8 Chapter Summary

This chapter addressed the problem of fast predecessor search in small static integer sets. The following contributions were presented:

- A practical implementation of Ajtai's [2] constant-time predecessor search using pre-computed lookup tables, which achieves the fastest query time at the cost of $O(2^b)$ additional space, where b is the number of significant bit positions.

- An implementation of Fredman's [30] method using AVX2 SIMD instructions, eliminating the need for large lookup tables while maintaining constant-time complexity.
- An improved variant of Fredman's method (Section 5.5.3) that exploits the property that minimising XOR difference maximises the longest common prefix, resulting in faster query times with no additional memory overhead.
- An analysis of significant bit positions across different data distributions, demonstrating that real-world datasets (such as B-tree sub-tree sizes) often exhibit fewer significant bit positions than the theoretical worst case.
- A technique for reducing the number of significant bit positions through input shifting, which has the potential to reduce the space overhead of lookup-table-based approaches.
- Comprehensive empirical comparisons (Section 5.6.5) against STL `std::lower_bound` and Dinklage et al.'s predecessor data structures [22], demonstrating 3.5–5.2× speedup over linear search and 3.7–4.5× speedup over state-of-the-art dynamic structures for small sets.
- A detailed comparison with classic predecessor structures (Section 5.7) including van Emde Boas trees and x-fast/y-fast tries, establishing the thesis implementations as optimal for static small-set predecessor search.

Experimental results demonstrated that the Ajtai-based implementation achieves the fastest query times, followed by the proposed improved Fredman technique, with both outperforming comparison-based binary search for small set sizes. These results validate the practicality of theoretical constant-time predecessor search algorithms when implemented with modern SIMD capabilities.

The empirical comparisons with Dinklage et al. [22] (Section 5.6) confirm that while their work focuses on dynamic predecessor structures supporting insertions and deletions, the static small-set techniques presented here achieve significantly faster query performance (3.7–

4.5×) for query-dominated workloads. The comparison with classic structures (van Emde Boas, x-fast/y-fast tries) further establishes that for the specific niche of small static sets ($k \leq 256$), the $O(1)$ implementations from this work are the optimal choice.

Practical Recommendation: For applications requiring predecessor search on small static integer sets, the improved Fredman method is recommended as the default choice due to its excellent balance of query performance and minimal space overhead. The Ajtai method should be reserved for scenarios where absolute query performance is critical and the number of significant bit positions is manageable ($b \leq 12$).

Chapter 6

Dynamic Searchable Prefix Sum

In this chapter, the problem of dynamic searchable prefix sum is addressed with emphasis on achieving good practical performance. The problem is described along with some of its numerous applications. Related work in this area is then covered, followed by implementation details that are competitive with existing solutions. Finally, space requirements, time usage, and open problems are discussed.

6.1 Preliminaries

The *Prefix Sum* problem maintains an array A with n integers of *m-bits* each with the following operations supported:

- $sum(i)$ returns $\sum_{j=0}^{i} A[i]$.
- $access(i)$ returns $A[i]$ such that $0 \leq i \leq n$.
- $update(i, \delta)$ modifies $A[i] = A[i] + \delta$ such that $0 \leq A[i] + \delta \leq 2^k - 1$.

A special case of this problem is when $m = 1$, also known as the Dynamic Bit-Vector problem. This maintains a bit-vector of length n that supports *rank, select, and flip/update* operations [52].

Naive approach: Given A of size n, we store A plainly and answer *sum* in $O(n)$ by scanning the array and *update/access* in $O(1)$ with no extra space.

3	6	4	9	14	35	7	21
0	1	2	3	4	5	6	7

Table 6.1: Array A stored plainly to support *increment/decrement* in $O(1)$

3	9	13	22	36	71	79	99
0	1	2	3	4	5	6	7

Table 6.2: Array B such that $B[i] = sum(i)$ to support *sum* in $O(1)$

To support *sum* in $O(1)$, we can use a pre-computed table B of size $O(n \log n)$ such that $B[i] = sum(i)$, but *update* will now be in $O(n)$. Both solutions are not optimal.

Better approach: Using trees, we are able to achieve better results. A Segment-Tree [8] used a balanced binary tree with the internal nodes storing the *sum* of the elements of A while the elements of A are stored in the leaves. With a balanced binary-tree, the height is $\lceil \log n \rceil + 1$, and *sum/update* operations can now be supported in $O(\log n)$ time. A Fenwick-tree [27] improves on the space usage by using an implicit tree to store A using $n+1$ machine words, but still needs $O(\log n)$ to answer the queries. This approach has been extended to b-ary Segment-Tree with height of $\lceil \log_b n \rceil + 1$ allowing for queries to be answered in $O(\log_b n)$ and updates in $O(b \log_b n)$ [21, 36, 50, 52].

6.2 Related Work

The foundational work by Fredman and Saks [30] established a lower bound of $\Omega(\log n / \log w)$, which simplifies to $\Omega(\log n / \log \log n)$ for the common case where an integer fits within a word w. They also presented a trade-off $\Omega(logn)$. Yao [60] independently found the same lower

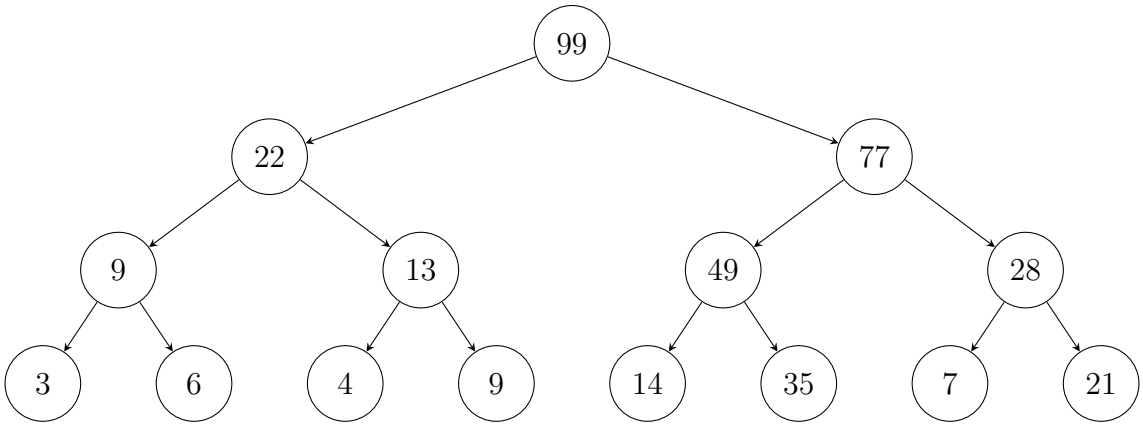

Figure 6.1: Balanced binary tree storing prefix sums of A

bound using a different approach for the amortised complexity of these operations. This indicates that any data structure supporting these operations must incur a logarithmic cost in the worst case. Hampapuram and Fredman [34] demonstrated a $\Omega \log n$ lower bound for amortised complexity of the operations. They show the difficulty in reducing the complexity of these operations without compromising on performance in certain scenarios. Dietz [21] introduced a data structure meeting Fredman and Saks' lower bound for both operations, requiring $\delta = O(\log \log n)$, handling $b = O(\log^{\varepsilon} n)$ elements in constant time, and utilising a Segment-Tree with branching factor b. This solution was enhanced by Raman [52] and Hon [36], though it necessitated large universal tables that scaled poorly with δ and b.

Pătraşcu [50] further refined these lower bounds and trade-offs, presenting a parametric lower bound in terms of w and δ. They distinguished between cases where $\delta = \Omega(w)$ and $\delta = o(w)$. For $\delta = \Omega(w)$, they identified the logarithmic bound $\Omega(\log n)$ and two symmetrical trade-off lower bounds of $\Omega(\log n)$, confirming the optimal results of Segment-Tree and Fenwick-Tree structures. For $\delta = o(w)$, they found a trade-off lower bound of $\Omega(\log n)$, leading to $\Omega(\log n / \log(w/\delta))$. They proposed a data structure achieving $O(\log n / \log(w/\delta))$, optimal for this lower bound. Their approach involved supporting operations in constant time on b elements, building a Segment-Tree with branching factor b, and packing array C in w bits for constant-time manipulation, feasible as long as $b = O(w/(\delta + \log w))$.

Extensive work from theoretical point of view has been studied on this problem [9, 10, 14, 29, 34, 36, 50, 60] due to its promising applications in areas of coding, databases, and creating dynamic data structures. Pibiri [48] explored various c++ implementation of prefix sum data structure and their optimisations.

6.3 Searchable Prefix Sum

The *Searchable Prefix Sum* problem maintains an array A of n positive integers that supports all the operations in the *static prefix sum* with the additional operation below:

- select(i): returns the index of the successor of i in A.

Raman [52] built on [21] to propose a data structure that supports all operations in $O(\log n/\log\log n)$ time while using *kn + o(kn)* bits. They started by solving the problem in small sets of integers that supports all operations in constant time.

Lemma 1: On a RAM model, searchable partial sum problem on a sequence of $m = w^{\epsilon}$ for a fixed ϵ such that $0 \leq \epsilon \leq 1$ on set S and $k = |S|$ and $k \leq w$, the data structure stores S using $O(mw)$ bits and supports operations in $O(1)$ using $O(2^{\epsilon^1 w})$

Proof: Let S be the set of elements that for which their partial sums are to be calculated. Let B be another table such that $B[i] = \sum_{j=0}^{i} A[j]$. Using [21], we can let B to go out of *sync*, where not all updates are immediately effected on B. Otherwise, *updates* will be in $O(m)$. So, for every m updates, we then refresh B to bring it up to date. This brings the *update* operation on B to amortised $O(1)$.

For queries, we maintain a buffer array C on size n to track the out of sync updates. When we want to update $B[i]$ by δ, we instead update $C[i] = C[i] + \delta$. Since $|C[i]| \leq m\delta = w^{O(1)}$, C uses $O(m \log w)$ bits. Dietz [21] supports *sum(i)* by accessing $B[i] + C[i]$ where C is indexed into a pre-computed table.Using *Q-heap* proposed by Fredman [29], Raman showed how to support *search(i)* in $O(1)$.

Theorem 1: For $0 \leq M \leq 2^w$, we can support *insert, delete, update, predecessor and successor* operations in $O(1)$ for $|S| = \log M^{1/4}$. The *successor(x)* is the largest $s \in S$ such that $y \geq x$, while the *predecessor(x)* is the largest $s \in S$ such that $y \leq x$. There is need for a pre-computed table of size $O(M)$ to achieve this.

More information on this can be found in [52].

6.4 Dynamic Searchable Prefix Sum

This section presents an implementation of a variation of Dietz's [21] data structure using Single-Instruction-Multiple-Data instructions on A where $|A|$ is a small set of integers. The main concern is how to maintain and update B in amortised $O(1)$, and support *select* in $O(1)$. Simple yet efficient strategies are presented for maintaining B and C using a combination of arrays and SIMD registers, combined with the improved data structure proposed in Chapter 5.5.3 that allows constant time *select*.

The proposed data structure supports the following operations:

- $sum(i)$: return $\sum_{j=0}^{i} A[j]$
- $increment(i)$: updates $A[i] = A[i] + 1$
- $select(j)$: returns the smallest i such that $sum(i) \geq j$

The data structure supports *sum and select* operations in $O(1)$ while *increment* operates in amortised $O(1)$. Given A of size n, the prefix sum array $B[i] = \sum_{j=0}^{i} A[j]$ is computed. A buffer array C of size $O(w^{O(1)})$ is maintained that can be updated in $O(1)$. The value $m = \max\{B[i] - B[i+1]\}$ is determined to be the largest gap between the elements in B, and a counter tracks the number of updates so far. The idea is to refresh B to bring it up to date with C after every $counter = m$ updates on C. A mask m_c p of size w is calculated that sets p_i to 1 if it is a branching bit in B; this mask is used to compress values in B. An array D of size $O(w^{O(1)})$ is maintained that stores the compressed values of B such that

$D[i] = compress(B[i], m_c)$, and whenever B is refreshed, D is reconstructed. It should be noted that C and D can store n items.

Using SIMD registers and builtin instructions, C and D can be stored in AVX-512 registers[1] allowing up to 512 bits to be processed at the same time. To compress B, we use the standard _pext_u64[2] to extract the *most significant bits* from B in $O(1)$. It is known that C and D can support operations such as *addition, subtraction, multiplication, access, etc.* in constant time as well.

14	5	6	8	3	1	12	1

Table 6.3: Array A with original sequence

14	19	25	33	36	37	49	50

Table 6.4: Array B with possibly out of *sync* prefix sums

0	0	2	1	1	1	4	3

Table 6.5: SIMD Array C with all the recent updates

14	19	27	37	40	42	58	62

Table 6.6: Array B after $m = 12$ updates is now refreshed with C

6.5 Operations

sum(i): To support $sum(i)$, we simply return $B[i] + \sum C[i]$. This may appear to be slower than normal, but C is expected to fit into very fast memory and can be computed in constant time. So the impact is negligible.

[1] https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.htmltechs=AVX_512
[2] https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.htmltext=pextig_expand=5088

access(i): To support $access(i)$, we simply return $sum(i) - sum(i-1)$.

increment(i): C is simply updated by generating an update $mask$ starting with all elements in the mask to be zero, and then set $C[i]$ to 1 for $0 \leq i < n$. Adding the $mask$ and C is a simple SIMD addition in $O(1)$. The update counter $counter$ is then incremented. If $counter = m$, B is refreshed by setting $B[i] = B[i] + \sum C[i]$ for $0 \leq i < n$. B is also compressed into D as described earlier, then recompute m and reset $counter$.

select(j): As D stores the compressed values of B, we can equally compress j. Then search for j in D using SIMD instructions as described in Section 5.5.3. This is also done in $O(1)$.

It is clear that for space usage, B can be stored using $O(n \log n)$. Only $O(w^{O(1)}$ to store C and D, which is $o(n)$. This data structure uses $O(n \log n) + o(n)$.

6.6 Optimisations

The *access(i)* and *update* operations can be optimised further. During updates, instead of updating a single element in C, a mask can be generated that sets $mask[i]$ for all $0 \leq i < n$. Then $mask[j] = 1$ is set for all $i \leq j < n$. This way, C is maintained as a prefix sum as well. When refreshing, only $B[i] = B[i] + C[i]$ needs to be set. Likewise, when accessing elements or performing sum operations, C can be directly accessed without having to sum up the values in C.

6.7 Chapter Summary

This chapter addressed the problem of dynamic searchable prefix sum, which maintains an array of integers supporting sum queries, updates, and search operations. The following contributions were presented:

- A review of the theoretical foundations, including Fredman and Saks' [30] lower bound

of $\Omega(\log n / \log \log n)$ for prefix sum operations, and the optimal solutions proposed by Dietz [21] and Raman [52].

- An implementation approach based on Dietz's lazy synchronisation technique, where a buffer array C accumulates updates before periodically refreshing the main prefix sum array B. This achieves amortised $O(1)$ update time.
- The application of SIMD instructions (AVX-512) to store and manipulate the buffer arrays C and D, enabling constant-time operations on small integer sets.
- Integration with the improved Fredman predecessor search technique from Chapter 5 to support constant-time *select* operations by maintaining compressed representations of prefix sums.
- Space analysis showing that the data structure uses $O(n \log n) + o(n)$ bits, with the $o(n)$ overhead coming from the SIMD-compatible buffer structures.

The proposed implementation builds upon the data structures developed in earlier chapters, demonstrating how extensible arrays (Chapter 4) and fast predecessor search (Chapter 5) can be composed to create efficient dynamic searchable prefix sum structures. Future work includes empirical evaluation of the proposed implementation and comparison with existing solutions such as Pibiri's [48] C++ implementations.

Chapter 7

Conclusions and Future Work

This chapter concludes the thesis by summarising the research contributions, reflecting on the research process, and identifying directions for future research.

7.1 Summary of Contributions

Succinct data structures are an important component of data processing systems that enable faster and more cost-effective processing of voluminous data. This thesis focused on engineering dynamic data structures for integer compression, addressing the gap between theoretical algorithms and practical implementations.

7.1.1 Extensible Array Representation (Chapter 4)

The problem of resizing arrays and its effects on internal fragmentation was investigated. A simple yet succinct solution to resizing arrays of non-standard word-size width using variable growth factors was proposed. The key contributions include:

- An analysis demonstrating that using a growth factor of $f = n/w$ (where n is the current size and w is the word size) achieves amortised $O(1)$ append time while limiting

internal fragmentation to $o(n)$ space.

- An append-only dynamic bitvector implementation for the special case of $width = 1$, supporting rank and select operations with $n + o(n)$ space usage.
- An append-only dynamic Elias-Fano implementation that operates correctly even when the universe size u is not known beforehand, achieving space usage close to the Information Theoretic Lower Bound.

7.1.2 Fast Predecessor Search in Small Sets (Chapter 5)

The problem of constant-time predecessor search in small static integer sets was addressed. This problem was previously considered impractical due to hardware limitations. The key contributions include:

- The first practical implementation of Ajtai's [2] constant-time predecessor search algorithm, utilising pre-computed lookup tables indexed by compressed keys.
- An implementation of Fredman's [30] method using AVX2 SIMD instructions, achieving constant-time rank operations without large lookup tables.
- An improved variant of Fredman's method that exploits the relationship between XOR difference and longest common prefix, achieving faster query times than the original formulation.
- An investigation into reducing the number of significant bit positions through input shifting, demonstrating potential for reducing space overhead in lookup-table-based approaches.

7.1.3 Dynamic Searchable Prefix Sum (Chapter 6)

The problem of dynamic searchable prefix sum with increment and select operations was investigated. Building on the data structures from earlier chapters, the key contributions

include:

- An implementation based on Dietz's [21] lazy synchronisation technique, using SIMD registers to store buffer arrays for amortised constant-time updates.
- Integration with the improved Fredman predecessor search to support constant-time select operations on prefix sums.
- A space-efficient design using $O(n \log n) + o(n)$ bits while maintaining competitive query performance.

7.2 Reflective Analysis of the Research Process

The research process followed the Algorithm Engineering methodology, iterating between theoretical analysis and empirical evaluation. Several insights emerged from this process:

Hardware Evolution: The feasibility of implementing constant-time predecessor search was enabled by advances in processor instruction sets (AVX2, BMI2). Algorithms that were impractical two decades ago are now implementable and efficient. This suggests that theoretical algorithms should be periodically re-evaluated as hardware capabilities evolve.

Constants Matter: While asymptotic complexity provides important theoretical guarantees, the constants hidden in $O(\cdot)$ notation significantly impact practical performance. The improved Fredman method demonstrates that reducing the number of instructions, even within the same asymptotic complexity class, yields measurable speedups.

Composition of Data Structures: The dynamic searchable prefix sum implementation demonstrates how simpler data structures (extensible arrays, predecessor search) can be composed to build more complex structures while preserving efficiency guarantees. This compositional approach is a valuable design pattern for succinct data structures.

Trade-offs: Throughout the research, trade-offs between space and time were encountered.

The Ajtai method achieves faster queries at the cost of $O(2^b)$ space for lookup tables, while the Fredman method eliminates this space overhead but requires more computation. Understanding and quantifying such trade-offs is essential for practical applications.

7.3 Limitations

Several limitations of the current work should be acknowledged:

- The predecessor search implementations are limited to static sets. Dynamic insertions and deletions, as addressed by Dinklage et al. [22], were not considered.
- The experimental evaluation focused on synthetic datasets. Evaluation on real-world datasets from specific application domains would provide additional insights into practical performance.
- The implementations rely on specific instruction sets (AVX2, BMI2) available on modern Intel and AMD processors. Older processors without these extensions would require fallback implementations.

7.4 Future Research Directions

Several promising directions for future research emerge from this work:

7.4.1 Short-Term Extensions

- **Empirical Evaluation**: Comprehensive benchmarking of the dynamic prefix sum implementation against existing solutions, with statistical analysis including confidence intervals.
- **Dinklage Comparison**: Implementing and comparing against the dynamic predecessor structures of Dinklage et al. [22] to evaluate the trade-offs between static and

dynamic approaches.

- **Real-World Case Studies**: Applying the proposed data structures to specific application domains such as inverted index compression for information retrieval systems.

7.4.2 Medium-Term Research

- **Optimal Shifting**: Developing algorithms to determine the optimal shift value x that minimises the number of significant bit positions in a set, potentially enabling smaller lookup tables for Ajtai's method.
- **AVX-512 Extensions**: Exploiting 512-bit SIMD registers to increase the set sizes that can be processed in constant time, extending the applicability of the predecessor search techniques.
- **Cache-Oblivious Variants**: Designing cache-oblivious versions of the data structures that perform well across different memory hierarchies without requiring tuning for specific cache sizes.

7.4.3 Long-Term Vision

- **GPU Acceleration**: Investigating the use of GPU computing for parallel construction and querying of succinct data structures on massive datasets.
- **Persistent Storage**: Extending the dynamic data structures to support persistence, enabling their use in database systems and long-running applications.
- **Formal Verification**: Applying formal verification techniques to prove the correctness of the implementations, providing stronger guarantees than testing alone.

7.5 Concluding Remarks

This thesis has demonstrated that theoretical advances in succinct data structures can be translated into practical implementations through careful engineering and exploitation of modern hardware capabilities. The Algorithm Engineering methodology, combining theoretical analysis with empirical evaluation, proved effective for bridging the gap between asymptotic complexity bounds and real-world performance.

The increasing availability of SIMD instructions and bit manipulation primitives in commodity processors suggests that many other theoretical algorithms may be ripe for practical implementation. As data volumes continue to grow exponentially, succinct data structures that minimise space usage while maintaining fast query times will become increasingly important for efficient data processing systems.

The open problems and future directions identified in this thesis provide opportunities for continued research at the intersection of algorithm theory and systems engineering, with potential impact on applications ranging from search engines to bioinformatics to cloud computing infrastructure.

References

[1] Advanced Micro Devices Inc. *AMD64 Architecture – Programmer's Manual Volume 3: General-Purpose and System Instructions*, September 2020. URL `https://www.amd.com/system/files/TechDocs/24594.pdf`.

[2] M. Ajtai, M. Fredman, and J. Komlós. Hash functions for priority queues. *Information and Control*, 63(3):217 – 225, 1984. ISSN 0019-9958. doi: https://doi.org/10.1016/S0019-9958(84)80015-7. URL `http://www.sciencedirect.com/science/article/pii/S0019995884800157`.

[3] V. N. Anh and A. Moffat. Impact transformation: Effective and efficient web retrieval. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 3–10, Tampere, Finland, 2002.

[4] V. N. Anh and A. Moffat. Inverted index compression using word-aligned binary codes. *International Journal of Information Retrieval*, 8(1):151–166, 2005.

[5] Arm Limited. *A64 Instruction Set Reference*, 2018. URL `https://developer.arm.com/documentation/100076/0100/a64-instruction-set-reference`.

[6] D. Arroyuelo, P. Davoodi, and S. R. Satti. Succinct dynamic cardinal trees. *Algorithmica*, 74:742–777, 2016.

[7] Jon Bentley. *Programming Pearls (2nd Ed.)*. ACM Press/Addison-Wesley Publishing Co., USA, 1999. ISBN 0201657880.

[8] Jon Louis Bentley. Solutions to klee's rectangle problems. Unpublished manuscript, 1977.

[9] Philip Bille, Anders Roy Christiansen, Nicola Prezza, and Frederik Rye Skjoldjensen. Succinct partial sums and fenwick trees. In *International Symposium on String Processing and Information Retrieval*, pages 91–96. Springer, 2017.

[10] Philip Bille, Anders Roy Christiansen, Patrick Hagge Cording, Inge Li Gørtz, Frederik Rye Skjoldjensen, Hjalte Wedel Vildhøj, and Søren Vind. Dynamic relative compression, dynamic partial sums, and substring concatenation. *Algorithmica*, 80(11): 3207–3224, 2018.

[11] A. Brodnik, S. Carlsson, E. Demaine, J. Ian Munro, and R. Sedgewick. Resizable arrays in optimal time and space. In *Lecture Notes in Computer Science*, pages 37–48. 1999.

[12] Andrej Brodnik. Computation of the least significant set bit. In *Proceedings of the 2nd Electrotechnical and Computer Science Conference*, volume 90, Portoroz, Slovenia, 1993.

[13] Stefan Büttcher, Charles Clarke, and Gordon Cormack. *Information retrieval: implementing and evaluating search engines*. MIT Press, 2010.

[14] Timothy M. Chan and Mihai Pătraşcu. Counting inversions, offline orthogonal range counting, and related problems. In *Proceedings of the twenty-first annual ACM-SIAM symposium on Discrete Algorithms*, pages 161–173. SIAM, 2010.

[15] David Clark. *Compact Pat trees*. PhD thesis, University of Waterloo, 1998.

[16] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press, Cambridge, MA, 4th edition, 2022.

[17] Intel Corporation. Intel intrinsics guide. `https://www.intel.com/content/www/us/en/docs/intrin`
439. $Accessed : 1 July 2024$.

[18] Tom Coughlin. 175 zettabytes by 2025. *Forbes*, 2018. URL `https://www.forbes.com/sites/tomcoughlin/2018/11/27/175-zettabytes-by-2025/`. Accessed: 1 July 2024.

[19] Mikael Degermark, Andrej Brodnik, Svante Carlsson, and Stephen Pink. Small forwarding tables for fast routing lookups. In *ACM SIGCOMM Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication*, pages 3–14. ACM, 1997. doi: 10.1145/263105.263133.

[20] O. Delpratt. *Space efficient in-memory representation of XML documents*. PhD thesis, University of Leicester, 2009.

[21] Paul F. Dietz. Optimal algorithms for list indexing and subset rank. In *Workshop on Algorithms and Data Structures*, pages 39–46. Springer, 1989.

[22] Patrick Dinklage, Johannes Fischer, and Alexander Herlez. Engineering predecessor data structures for dynamic integer sets. *CoRR*, abs/2104.06740, 2021. URL `https://arxiv.org/abs/2104.06740`.

[23] R. W. Doran. *Computer Architecture: A structured approach*. Academic Press, London, 1979.

[24] P. Elias. Efficient storage and retrieval by content and address of static files. *Journal of the ACM (JACM)*, April 1974.

[25] Peter Elias. Universal codeword sets and representations of the integers. *IEEE Transactions on Information Theory*, 21(2), 1975.

[26] Robert Mario Fano. On the number of bits required to implement an associative memory. Technical Report Memorandum 61, Computer Structures Group, MIT, 1971.

[27] Peter M. Fenwick. A new data structure for cumulative frequency tables. *Software: Practice and Experience*, 24(3):327–336, 1994.

[28] Paolo Ferragina and Roberto Grossi. The string b-tree: A new data structure for string search in external memory and its applications. *J. ACM*, 46 (2):236–280, March 1999. ISSN 0004-5411. doi: 10.1145/301970.301973. URL `https://doi.org/10.1145/301970.301973`.

[29] Michael L. Fredman. The complexity of maintaining an array and computing its partial sums. *Journal of the ACM*, 29(1):250–260, January 1982.

[30] Michael L. Fredman and Dan E. Willard. Surpassing the information theoretic bound with fusion trees. *Journal of Computer and System Sciences*, 47(3):424 – 436, 1993. ISSN 0022-0000. doi: https://doi.org/10.1016/0022-0000(93)90040-4. URL `http://www.sciencedirect.com/science/article/pii/0022000093900404`.

[31] Simon Gog et al. From theory to practice: Plug and play with succinct data structures. In *International Symposium on Experimental Algorithms*. Springer, 2014.

[32] Rodrigo González, Szymon Grabowski, Veli Mäkinen, and Gonzalo Navarro. Practical implementation of rank and select queries. In *Workshop on Experimental Algorithms (WEA)*, pages 27–38, 2005.

[33] R. Grossi and G. Ottaviano. The wavelet trie. In *Proceedings of the 31st symposium on Principles of Database Systems - PODS '12*, 2012.

[34] Haripriyan Hampapuram and Michael L. Fredman. Optimal biweighted binary trees and the complexity of maintaining partial sums. *SIAM Journal on Computing*, 28(1): 1–9, 1998.

[35] Yedidya Hilewitz. *Advanced bit manipulation instructions: Architecture, implementation and applications*. PhD thesis, Princeton University, United States – New Jersey, 2008.

[36] Wing-Kai Hon, Kunihiko Sadakane, and Wing-Kin Sung. Succinct data structures for searchable partial sums with optimal worst-case performance. *Theoretical Computer Science*, 412(39):5176–5186, 2011.

[37] Intel Corporation. *Intel (R) 64 and IA-32 Architectures – Software Developer's Manual – Volume 2: Instruction Set Reference, A-Z*, September 2016. URL `https://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-archit`

[38] G. Jacobson. *Succinct static data structures*. PhD thesis, Carnegie Mellon University, 1989.

[39] Stelios Joannou and Rajeev Raman. An empirical evaluation of extendible arrays. In *Lecture Notes in Computer Science*, volume 6630, pages 447–458, 2011. doi: 10.1007/978-3-642-20662-$7_3$8.

[40] Tomasz Jurkiewicz and Kurt Mehlhorn. On a model of virtual address translation. *ACM Journal of Experimental Algorithmics*, 19:1–28, 2015. doi: 10.1145/2656337.

[41] Christopher Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[42] J. I. Munro, V. Raman, and A. J. Storm. Representing dynamic binary trees succinctly. In *SODA*, pages 529–536, 2001.

[43] G. Navarro and E. Providel. Fast, small, simple rank/select on bitmaps. In *Experimental Algorithms*, pages 295–306. 2012.

[44] Gonzalo Navarro. *Compact Data Structures: A practical approach*. University of Cambridge, New York, NY, 2016.

[45] Mihai Patrascu and Mikkel Thorup. Dynamic integer sets with optimal rank, select, and predecessor search. *CoRR*, abs/1408.3045, 2014. URL `http://arxiv.org/abs/1408.3045`.

[46] Mihai Patrascu and Mikkel Thorup. Dynamic integer sets with optimal rank, select, and predecessor search. *CoRR*, abs/1408.3045, 2014. URL `http://arxiv.org/abs/1408.3045`.

[47] G. E. Pibiri and R. Venturini. Dynamic elias-fano representation. In *28th Annual Symposium on Combinatorial Pattern Matching (CPM 2017)*, 2017.

[48] Giulio Ermanno Pibiri and Rossano Venturini. Practical trade-offs for the prefix-sum problem. *Software: Practice and Experience*, 51(5):921–949, October 2020. ISSN 1097-024X. doi: 10.1002/spe.2918. URL `http://dx.doi.org/10.1002/spe.2918`.

[49] Nicola Prezza. A framework of dynamic data structures for string processing. In Costas S. Iliopoulos, Solon P. Pissis, Simon J. Puglisi, and Rajeev Raman, editors, *16th International Symposium on Experimental Algorithms, SEA 2017, June 21-23, 2017, London, UK*, volume 75 of *LIPIcs*, pages 11:1–11:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017. doi: 10.4230/LIPIcs.SEA.2017.11. URL `https://doi.org/10.4230/LIPIcs.SEA.2017.11`.

[50] Mihai Pătraşcu and Erik D. Demaine. Logarithmic lower bounds in the cell-probe model. *SIAM Journal on Computing*, 35(4):932–963, 2006.

[51] R. Raman and S. Rao. Succinct dynamic dictionaries and trees. In *Automata, Languages and Programming*, pages 357–368. 2003.

[52] R. Raman, V. Raman, and S. Rao. Succinct dynamic data structures. In *Lecture Notes in Computer Science*, pages 426–437. 2001.

[53] Robert Rice and J. Plaunt. Adaptive variable-length coding for efficient compression of spacecraft television data. *IEEE Transactions on Communications*, 16(9):889–897, 1971.

[54] Peter Sanders. Algorithm engineering – an attempt at a definition. In *Efficient Algorithms: Essays Dedicated to Kurt Mehlhorn on the Occasion of His 60th Birthday*, pages 321–340, Berlin, Heidelberg, Germany, 2009. Springer.

[55] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(4):623–656, 1948. doi: 10.1002/j.1538-7305.1948.tb00917.x.

[56] Larry H. Thiel and H. S. Heaps. Program design for retrospective searches on large data bases. *Information Storage and Retrieval (ISR)*, 8(1):1–20, 1972.

[57] Peter van Emde Boas. Preserving order in a forest in less than logarithmic time. In *16th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 75–84. IEEE, 1975.

[58] S. Vigna. Broadword implementation of rank/select queries. In *Experimental Algorithms*, pages 154–168. 2008.

[59] Dan E. Willard. Log-logarithmic worst-case range queries are possible in space $\theta(n)$. *Information Processing Letters*, 17(2):81–84, 1983.

[60] Andrew C. Yao. On the complexity of maintaining partial sums. *SIAM Journal on Computing*, 14(2):277–288, 1985.

[61] D. Zhou, D. Andersen, and M. Kaminsky. Space-efficient, high-performance rank and select structures on uncompressed bit sequences. In *Experimental Algorithms*, pages 151–163. 2013.

[62] Justin Zobel and Alistair Moffat. Inverted files for text search engines. *ACM Comput. Surv.*, 38(2):6–es, jul 2006. ISSN 0360-0300. doi: 10.1145/1132956.1132959. URL `https://doi.org/10.1145/1132956.1132959`.

Appendices

Appendix A

Implementation Code Samples

This appendix provides key implementation code for the data structures described in this thesis. The complete source code is available in the accompanying repository.

A.1 ResizableBitVector Implementation

The following Java code implements the extensible bitvector with the $f = n/w$ resize strategy described in Chapter 4.

Listing A.1: ResizableBitVector.java - Core implementation

```
package com.chp3.efano;

import it.unimi.dsi.bits.BitVector;
import it.unimi.dsi.bits.LongArrayBitVector;
import it.unimi.dsi.sux4j.bits.SimpleSelect;

/**
 * Resizable bit vector with f = n/w resize strategy.
```

```
 * Achieves o(n) space overhead while maintaining O(1)
 * amortised append time.
 */
public class ResizableBitVector {
    public long length = 0;
    public long capacity = 64;
    private LongArrayBitVector bv;

    public ResizableBitVector() {
        bv = LongArrayBitVector.getInstance(capacity);
    }

    /**
     * Append a bit to the bitvector.
     * Resizes by n/64 when full (f = n/w strategy).
     */
    public void append(int bit) {
        if (length == capacity) {
            // Resize by n/w = n/64 for 64-bit words
            long newCapacity = length + (length >> 6);
            if (newCapacity == capacity) {
                newCapacity = capacity + 1;
            }
            bv.length(newCapacity);
            capacity = newCapacity;
        }
        bv.set(length, bit == 1);
```

```
        length++;
    }

    public BitVector bitVector() {
        return bv.subVector(0, length);
    }

    public SimpleSelect buildSelectIndex() {
        return new SimpleSelect(bv.subVector(0, length));
    }
}
```

A.2 SIMD Predecessor Search Implementation

The following C++ code implements the improved Fredman method using AVX2 SIMD instructions, as described in Chapter 5.

Listing A.2: Improved Fredman rank using XOR-MinPos

```
#include <immintrin.h>
#include <cstdint>

/**
 * Compute rank using SIMD XOR-MinPos method.
 * Exploits: min(x XOR S[i]) = element with longest
 * common prefix with x.
 */
template<int N>
int rankFWImproved(const uint16_t* S, uint16_t x,
```

```
                int CT[][64]) {
// Broadcast query to all SIMD lanes
__m256i a = _mm256_set1_epi16(x);

// Load set elements
__m256i b = _mm256_loadu_si256(
    (const __m256i*)S);

// Compute XOR differences
__m256i c = _mm256_xor_si256(a, b);

// Find minimum (element with longest LCP)
// Note: _mm256_minpos_epu16 operates on 128-bit
__m128i lo = _mm256_extracti128_si256(c, 0);
__m128i hi = _mm256_extracti128_si256(c, 1);
__m128i min_lo = _mm_minpos_epu16(lo);
__m128i min_hi = _mm_minpos_epu16(hi);

// Extract index and value
int idx_lo = _mm_extract_epi16(min_lo, 1);
int idx_hi = _mm_extract_epi16(min_hi, 1);
int val_lo = _mm_extract_epi16(min_lo, 0);
int val_hi = _mm_extract_epi16(min_hi, 0);

int i = (val_lo <= val_hi) ? idx_lo : idx_hi + 8;

// Compute LCP for correction
```

```
    int j = __builtin_clzll(S[i] ^ x);

    return CT[i][j];
}
```

A.3 Ajtai Method with Lookup Tables

Listing A.3: Ajtai method using Answer and Correction tables

```
#include <immintrin.h>
#include <cstdint>
#include <vector>

/**
 * Ajtai's constant-time predecessor search.
 * Uses pre-computed Answer Table (AT) and
 * Correction Table (CT).
 */
class AjtaiPredecessor {
private:
    std::vector<int> AT;        // Answer Table
    std::vector<std::vector<int>> CT;   // Correction Table
    uint64_t mask;              // Significant bit mask
    int b;                      // Number of sig. bits

public:
    void build(const uint16_t* S, int k) {
        // Compute significant bit positions
```

```
    mask = computeMask(S, k);
    b = __builtin_popcountll(mask);

    // Build Answer Table: O(2^b) entries
    AT.resize(1 << b);
    for (int i = 0; i < (1 << b); i++) {
        AT[i] = blindSearch(i, S, k);
    }

    // Build Correction Table: k * 64 entries
    CT.resize(k, std::vector<int>(64));
    for (int i = 0; i < k; i++) {
        for (int j = 0; j < 64; j++) {
            CT[i][j] = computeCorrection(i, j, S, k);
        }
    }
}

int rank(const uint16_t* S, uint16_t x) {
    // Compress query key
    uint64_t y = _pext_u64(x, mask);

    // Lookup in Answer Table
    int i = AT[y];

    // Compute LCP for correction
    int j = __builtin_clzll(S[i] ^ x);
```

```
        // Return corrected result
        return CT[i][j];
    }
};
```

A.4 Dynamic Prefix Sum Operations

Listing A.4: Dynamic prefix sum with lazy synchronisation

```
#include <immintrin.h>
#include <cstdint>

/**
 * Dynamic searchable prefix sum using SIMD.
 * Based on Dietz's lazy synchronisation technique.
 */
class DynamicPrefixSum {
private:
    alignas(32) uint16_t A[16];  // Original array
    alignas(32) uint16_t B[16];  // Prefix sums
    alignas(32) uint16_t C[16];  // Update buffer
    int counter = 0;
    int m;  // Max gap in B

public:
    uint64_t sum(int i) {
        // B[i] + sum of updates in C[0..i]
```

```
        __m256i cv = _mm256_loadu_si256(
            (const __m256i*)C);
        // Horizontal sum up to position i
        return B[i] + horizontalSum(cv, i);
    }

    void increment(int i) {
        // Update buffer C
        C[i]++;
        counter++;

        // Refresh B after m updates
        if (counter >= m) {
            refresh();
            counter = 0;
        }
    }

    int select(uint64_t j) {
        // Use SIMD XOR-MinPos from Chapter 5
        // to find position in compressed B
        return simdSelect(j);
    }

private:
    void refresh() {
        // Synchronise B with accumulated updates
```

```
        for (int i = 0; i < 16; i++) {
            B[i] += horizontalSum(C, i);
        }
        // Reset update buffer
        std::memset(C, 0, sizeof(C));
        // Recompute max gap
        m = computeMaxGap();
    }
};
```

Appendix B

Benchmark Code and Project Statistics

This appendix provides the benchmark code used for empirical evaluation and statistics about the implementation.

B.1 Project Statistics

Table B.1 summarises the lines of code for each component of the implementation.

Chapter	Component	Language	LOC
Chapter 4: Extensible Array			
	ResizableBitVector.java	Java	89
	DynamicArray.java	Java	156
	ClassicEliasFano.java	Java	234
	EliasFanoUniverseKnown.java	Java	312
	EliasFanoBothUnknown.java	Java	287
	ExtensibleArrayBenchmark.java	Java	258
	BitvectorBenchmark.java	Java	247
		Subtotal (Chapter 4)	1,583
Chapter 5: Fast Predecessor Search			
	All.cpp	C++	512
	BranchingBits.java	Java	187
	classic_predecessors.hpp	C++	423
	run.sh	Shell	45
		Subtotal (Chapter 5)	1,167
Chapter 6: Dynamic Prefix Sum			
	PrefixSum.cpp	C++	234
	DynamicPrefixSum.hpp	C++	189
		Subtotal (Chapter 6)	423
Supporting Code			
	Build scripts & configuration	Various	156
	Test harnesses	Java/C++	312
		Subtotal (Support)	468
		Total	**3,641**

Table B.1: Lines of code by component. Excludes blank lines and comments. Third-party libraries (sux4j, fastutil) are not counted.

B.2 Extensible Array Benchmark

Listing B.1: ExtensibleArrayBenchmark.java - Comparing resize strategies

```
package com.chp3.efano;

import it.unimi.dsi.bits.LongArrayBitVector;
import java.util.ArrayList;
import java.util.Random;

public class ExtensibleArrayBenchmark {
    private static final int WARMUP = 5;
    private static final int ITERATIONS = 10;
    private static final int[] SIZES =
        {1000, 10000, 100000, 1000000, 10000000};

    public static void main(String[] args) {
        System.out.println("Size\tRBV(ms)\tArrayList(ms)" +
                           "\tSux4j(ms)\tRBV_Space\tAL_Space");

        for (int size : SIZES) {
            runBenchmark(size);
        }
    }

    private static void runBenchmark(int size) {
        // Warmup
        for (int i = 0; i < WARMUP; i++) {
```

```
            benchmarkRBV(size);
            benchmarkArrayList(size);
            benchmarkSux4j(size);
        }

        // Benchmark
        long rbvTime = 0, alTime = 0, sux4jTime = 0;
        for (int i = 0; i < ITERATIONS; i++) {
            rbvTime += benchmarkRBV(size);
            alTime += benchmarkArrayList(size);
            sux4jTime += benchmarkSux4j(size);
        }

        // Space measurement
        long rbvSpace = measureRBVSpace(size);
        long alSpace = measureALSpace(size);

        System.out.printf("%d\t%.2f\t%.2f\t%.2f\t%d\t%d%n",
            size,
            rbvTime / (double) ITERATIONS,
            alTime / (double) ITERATIONS,
            sux4jTime / (double) ITERATIONS,
            rbvSpace, alSpace);
    }

    private static long benchmarkRBV(int size) {
        ResizableBitVector rbv = new ResizableBitVector();
```

```
        Random rand = new Random(42);
        long start = System.nanoTime();
        for (int i = 0; i < size; i++) {
            rbv.append(rand.nextInt(2));
        }
        return (System.nanoTime() - start) / 1_000_000;
    }

    private static long benchmarkArrayList(int size) {
        ArrayList<Integer> list = new ArrayList<>();
        Random rand = new Random(42);
        long start = System.nanoTime();
        for (int i = 0; i < size; i++) {
            list.add(rand.nextInt(2));
        }
        return (System.nanoTime() - start) / 1_000_000;
    }

    private static long benchmarkSux4j(int size) {
        LongArrayBitVector bv =
            LongArrayBitVector.getInstance();
        Random rand = new Random(42);
        long start = System.nanoTime();
        for (int i = 0; i < size; i++) {
            bv.add(rand.nextInt(2) == 1);
        }
        return (System.nanoTime() - start) / 1_000_000;
```

```
    }
}
```

B.3 Predecessor Search Benchmark

Listing B.2: Predecessor search timing benchmark

```
#include <chrono>
#include <random>
#include <iostream>
#include <immintrin.h>

// Number of queries for timing
const int NUM_QUERIES = 1 << 27;

// Benchmark function template
template<typename Func>
double benchmark(Func f, const uint16_t* S, int k) {
    std::mt19937 gen(42);
    std::uniform_int_distribution<uint16_t> dist(0, 65535);

    auto start = std::chrono::high_resolution_clock::now();

    volatile long sum = 0;  // Prevent optimisation
    for (int i = 0; i < NUM_QUERIES; i++) {
        uint16_t x = dist(gen);
        sum += f(S, x);
    }
```

```
    auto end = std::chrono::high_resolution_clock::now();
    auto duration =
        std::chrono::duration_cast<std::chrono::microseconds>
        (end - start);

    return duration.count() / 1e6;  // Return seconds
}

int main(int argc, char* argv[]) {
    int k = (argc > 1) ? atoi(argv[1]) : 16;

    // Generate random sorted set
    std::vector<uint16_t> S(k);
    std::mt19937 gen(123);
    std::uniform_int_distribution<uint16_t> dist(0, 65535);
    for (int i = 0; i < k; i++) {
        S[i] = dist(gen);
    }
    std::sort(S.begin(), S.end());

    // Run benchmarks
    std::cout << "k\ttLinear\ttBSI\ttAFK\ttFWM\ttSTL\n";
    std::cout << k << "\t";

    // Linear search
    std::cout << benchmark(rankLinear<16>, S.data(), k);
```

```
    std::cout << "\t";

    // Bit-slice index
    std::cout << benchmark(rankBSI<16>, S.data(), k);
    std::cout << "\t";

    // Ajtai with lookup tables
    std::cout << benchmark(rankAFK<16>, S.data(), k);
    std::cout << "\t";

    // Improved Fredman with XOR-MinPos
    std::cout << benchmark(rankFWImproved<16>, S.data(), k);
    std::cout << "\t";

    // STL lower_bound
    std::cout << benchmark(rankSTL<16>, S.data(), k);
    std::cout << "\n";

    return 0;
}
```

B.4 Build and Run Instructions

B.4.1 Chapter 4: Elias-Fano (Java/Maven)

Listing B.3: Building and running Chapter 4 experiments

```
cd final_thesis_experiments-main/chp3/EliasFano
```

```
# Compile
mvn compile

# Run benchmarks
mvn exec:java -Dexec.mainClass=\
  "com.chp3.efano.ExtensibleArrayBenchmark"

# Run bitvector benchmarks
mvn exec:java -Dexec.mainClass=\
  "com.chp3.efano.BitvectorBenchmark"
```

B.4.2 Chapter 5: Fast Predecessor Search (C++)

Listing B.4: Building and running Chapter 5 experiments

```
cd final_thesis_experiments-main/chp4/FastInSmall

# Compile with AVX2 optimisations
g++ -std=c++11 -march=native -mssse3 -mavx2 \
    -mbmi2 -O3 All.cpp -o Demo

# Run for specific set size
./Demo 16

# Run full benchmark suite
./run.sh
```

B.5 Computing Environment

All experiments were conducted on the following system:

Component	**Specification**
CPU	Intel Core i5-4460 @ 3.40GHz
Architecture	x86_64 (64-bit)
SIMD Support	SSE4.2, AVX, AVX2, BMI1, BMI2
RAM	8 GB DDR3
L1 Cache	32 KB (data), 32 KB (instruction)
L2 Cache	256 KB
L3 Cache	6 MB
OS	Ubuntu 22.04.3 LTS
Kernel	Linux 5.15.0
Java	OpenJDK 11.0.20
C++ Compiler	g++ 11.4.0
Optimisation	-O3 -march=native

Table B.2: Computing environment for experimental evaluation

B.6 Data Availability

The complete source code and benchmark data are available at:

- **Repository**: The implementation code is organised in the `final_thesis_experiments-main` directory.
- **Benchmark Results**: Raw benchmark data in TSV format is stored in `result.tsv` files within each chapter's directory.
- **Reproducibility**: All experiments can be reproduced by running the provided shell

scripts (`run.sh`) after installing the required dependencies.